```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load data
df = pd.read_csv("/large_sales_data.csv") # Updated path

# Features (removed 'Profit' to prevent data leakage)
X = df[['Revenue', 'Units_Sold', 'Marketing_Spend']]
# Placeholder for 'is_closed' if it doesn't exist. Creating a binary
target based on 'Profit' above average.
y = (df['Profit'] > df['Profit'].mean()).astype(int)

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

# Model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Predictions
df['win_probability'] = model.predict_proba(X)[:,1]

# Save output
df.to_csv("sales_predictions.csv", index=False)
```

Feature Engineering

Let's create some new features that might be predictive of our target variable. We will derive:

- **Profit Margin**: The percentage of revenue that turns into profit.
- **Marketing Efficiency**: Revenue generated per unit of marketing spend.
- **Average Unit Price**: The average price per unit sold.

```python
# Create Profit Margin
df['Profit_Margin'] = df['Profit'] / df['Revenue']

# Create Marketing Efficiency (handling division by zero for
Marketing_Spend)
df['Marketing_Efficiency'] = df.apply(lambda row: row['Revenue'] /
row['Marketing_Spend'] if row['Marketing_Spend'] > 0 else 0, axis=1)

# Create Average Unit Price (handling division by zero for Units_Sold)
df['Average_Unit_Price'] = df.apply(lambda row: row['Revenue'] /
row['Units_Sold'] if row['Units_Sold'] > 0 else 0, axis=1)

print("New features added:")
display(df[['Profit_Margin', 'Marketing_Efficiency',
'Average_Unit_Price']].head())
```

```
New features added:
```


```
{"summary":"{\n  \"name\": \"display(df[['Profit_Margin',
'Marketing_Efficiency', 'Average_Unit_Price']]\",\n  \"rows\": 5,\n
\"fields\": [\n  {\n  \"column\": \"Profit_Margin\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
0.5971357463092009,\n  \"min\": 0.06356766521150083,\n
\"max\": 1.4695984904077997,\n  \"num_unique_values\": 5,\n
\"samples\": [\n  0.4897234559731219,\n
0.1434009454705365,\n  1.4695984904077997\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"Marketing_Efficiency\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
13.77820212379787,\n  \"min\": 1.2937745829377458,\n
\"max\": 36.276758409785934,\n  \"num_unique_values\": 5,\n
\"samples\": [\n  9.519869154349406,\n
5.4495618962031,\n  1.2937745829377458\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"Average_Unit_Price\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
812.4183277441518,\n  \"min\": 46.99014778325123,\n
\"max\": 1819.2,\n  \"num_unique_values\": 5,\n
\"samples\": [\n  104.76933333333334,\n  1819.2,\n
46.99014778325123\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  }\n  ]\n}","type":"dataframe"}
```


Now, we need to update our feature set `X` to include these new engineered features before retraining the model.

```
# Update features to include engineered variables
X = df[['Revenue', 'Units_Sold', 'Marketing_Spend', 'Profit_Margin',
'Marketing_Efficiency', 'Average_Unit_Price']]

# Re-split the data with the new features
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42) # Added random_state for
reproducibility

print("Updated X_train features:")
display(X_train.head())
```

```
Updated X_train features:
```


```
{"summary":"{\n  \"name\": \"display(X_train\",\n  \"rows\": 5,\n
\"fields\": [\n  {\n  \"column\": \"Revenue\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
57126,\n  \"min\": 51321,\n  \"max\": 196321,\n
\"num_unique_values\": 5,\n  \"samples\": [\n  196321,\n
51321,\n  177296\n  ],\n  \"semantic_type\":
\"\",\n  \"description\": \"\"\n  }\n  },\n  {\n
```

```
\"column\": \"Units_Sold\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 390,\n        \"min\": 31,\n
\"max\": 949,\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          31,\n          877,\n          910\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Marketing_Spend\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
4318,\n        \"min\": 8496,\n        \"max\": 19147,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          19147,\n
8496,\n          12023\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Profit_Margin\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.057731474531001414,\n        \"min\":
0.06306508218682667,\n        \"max\": 0.20700415210896755,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
0.06306508218682667,\n          0.18715535550749207,\n
0.17853758686039167\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Marketing_Efficiency\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 3.232720520777776,\n        \"min\":
6.040607344632768,\n        \"max\": 14.746402728104467,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
10.253355617067948,\n          6.040607344632768,\n
14.746402728104467\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Average_Unit_Price\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 2750.983865070728,\n        \"min\":
58.5188141391106,\n        \"max\": 6332.935483870968,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
6332.935483870968,\n          58.5188141391106,\n
194.83076923076922\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Retraining Base Model with New Features

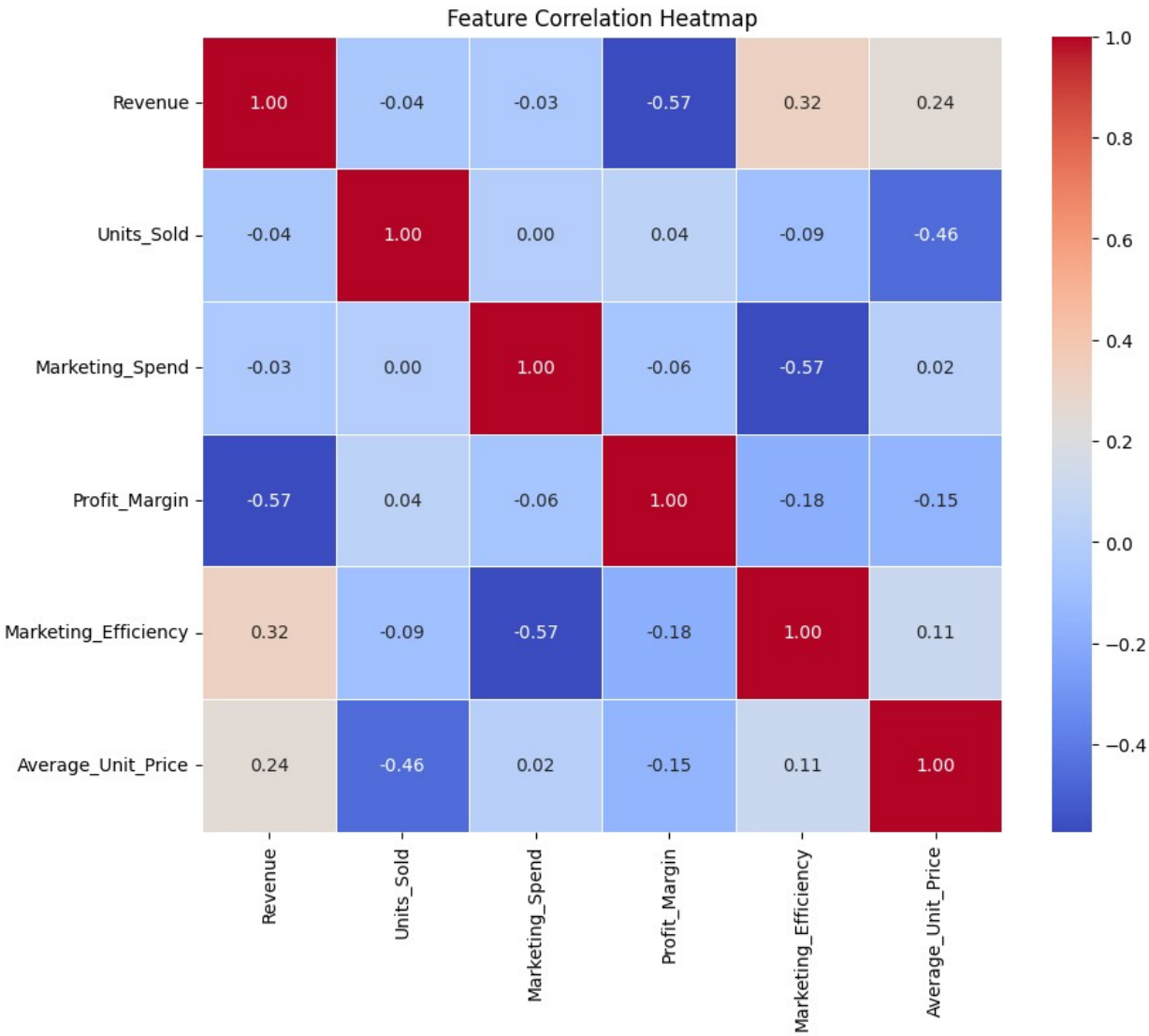
Now that our feature set `X` has been updated with the engineered variables, we need to retrain our initial Random Forest model (`model`) to leverage these new insights.

```
# Retrain the initial model with the updated X_train and y_train
model = RandomForestClassifier(n_estimators=100, random_state=42) #
Ensure reproducibility
model.fit(X_train, y_train)

print("Base model retrained with new features.")
```

Feature Correlation Analysis

To understand the relationships between our features, let's visualize their correlations using a heatmap. This can help identify highly correlated features which might be redundant or indicative of multicollinearity.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate the correlation matrix for the features in X_train
correlation_matrix = X_train.corr()

# Plot the heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
fmt=".2f", linewidths=.5)
plt.title('Feature Correlation Heatmap')
plt.show()
```



Addressing Multicollinearity

From the correlation heatmap, we observed a high positive correlation between 'Revenue' and 'Marketing_Efficiency'. High correlation between independent variables (multicollinearity) can negatively impact model stability and interpretability. To mitigate this, we will drop 'Marketing_Efficiency'.

```
import pandas as pd

df = pd.read_csv("/large_sales_data.csv")

# Remove general duplicate rows
df.drop_duplicates(inplace=True)

# Standardize categorical columns (assuming these columns exist)
for col in ['Region', 'Salesperson', 'Category', 'Product_Type']:
```

```
    if col in df.columns:
        df[col] = df[col].astype(str).str.lower().str.strip()

# Ensure numerical columns are non-negative (assuming these columns
exist)
for col in ['Revenue', 'Units_Sold', 'Profit', 'Marketing_Spend']:
    if col in df.columns:
        df = df[df[col] >= 0]

print("Data cleaning steps applied. Checking for remaining null
values:")
print(df.isnull().sum())
```

```
Data cleaning steps applied. Checking for remaining null values:
Region                    0
Salesperson               0
Revenue                   0
Profit                    0
Units_Sold                0
Customer_Satisfaction     0
Marketing_Spend           0
Discount_Percentage       0
Product_Category          0
Sales_Channel             0
Order_Processing_Time     0
dtype: int64
```

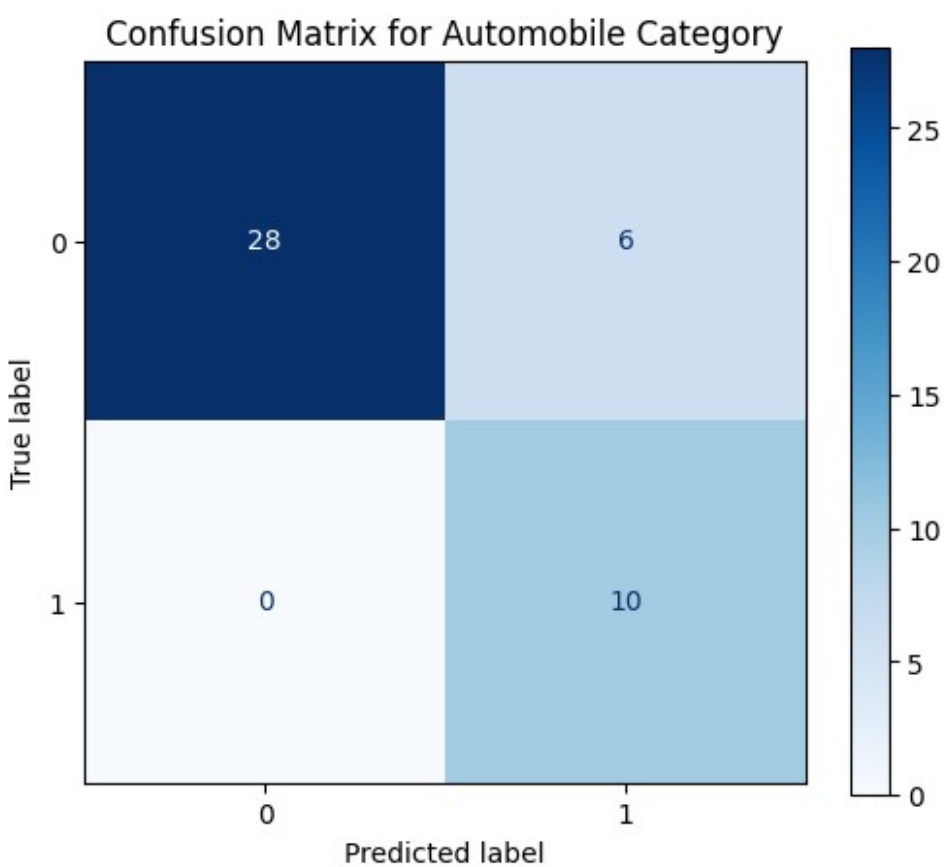
Error Distribution Comparison: 'Automobile' vs. 'Clothing' Categories

To understand the specific types of errors (False Positives and False Negatives) the model makes in the 'Automobile' and 'Clothing' categories, we will generate and visualize their confusion matrices.

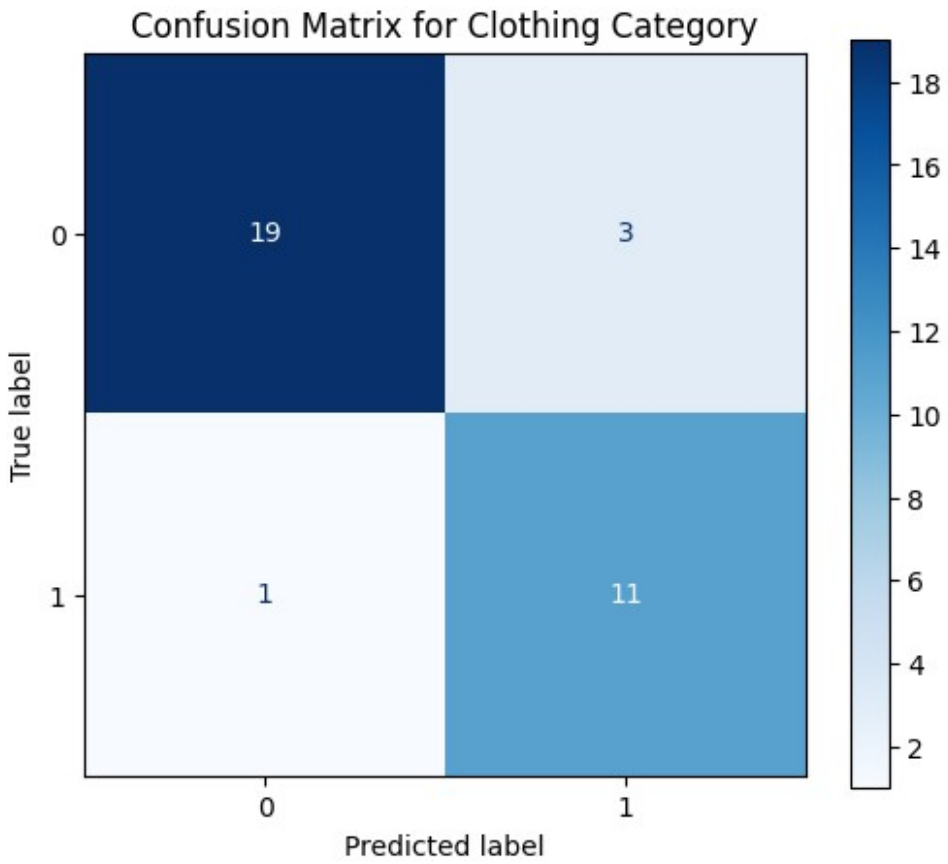
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

def plot_confusion_matrix_for_category(category_name, df_subset):
    """Plots the confusion matrix for a given product category
subset."""
    if df_subset.empty:
        print(f"No test data for Product Category: {category_name}")
        return

    true_labels = df_subset['true_label']
    predicted_labels = df_subset['predicted_label']

    cm = confusion_matrix(true_labels, predicted_labels)
    fig, ax = plt.subplots(figsize=(6, 5))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
```

```
display_labels=best_model.classes_)
    disp.plot(cmap=plt.cm.Blues, ax=ax)
    plt.title(f'Confusion Matrix for {category_name} Category')
    plt.show()


# Filter and plot for 'Automobile' category
automobile_df = test_df[test_df['Product_Category'] == 'Automobile']
print("\n--- Confusion Matrix for Automobile Category ---")
plot_confusion_matrix_for_category('Automobile', automobile_df)

# Filter and plot for 'Clothing' category
clothing_df = test_df[test_df['Product_Category'] == 'Clothing']
print("\n--- Confusion Matrix for Clothing Category ---")
plot_confusion_matrix_for_category('Clothing', clothing_df)
```

```
--- Confusion Matrix for Automobile Category ---
```




```
--- Confusion Matrix for Clothing Category ---
```

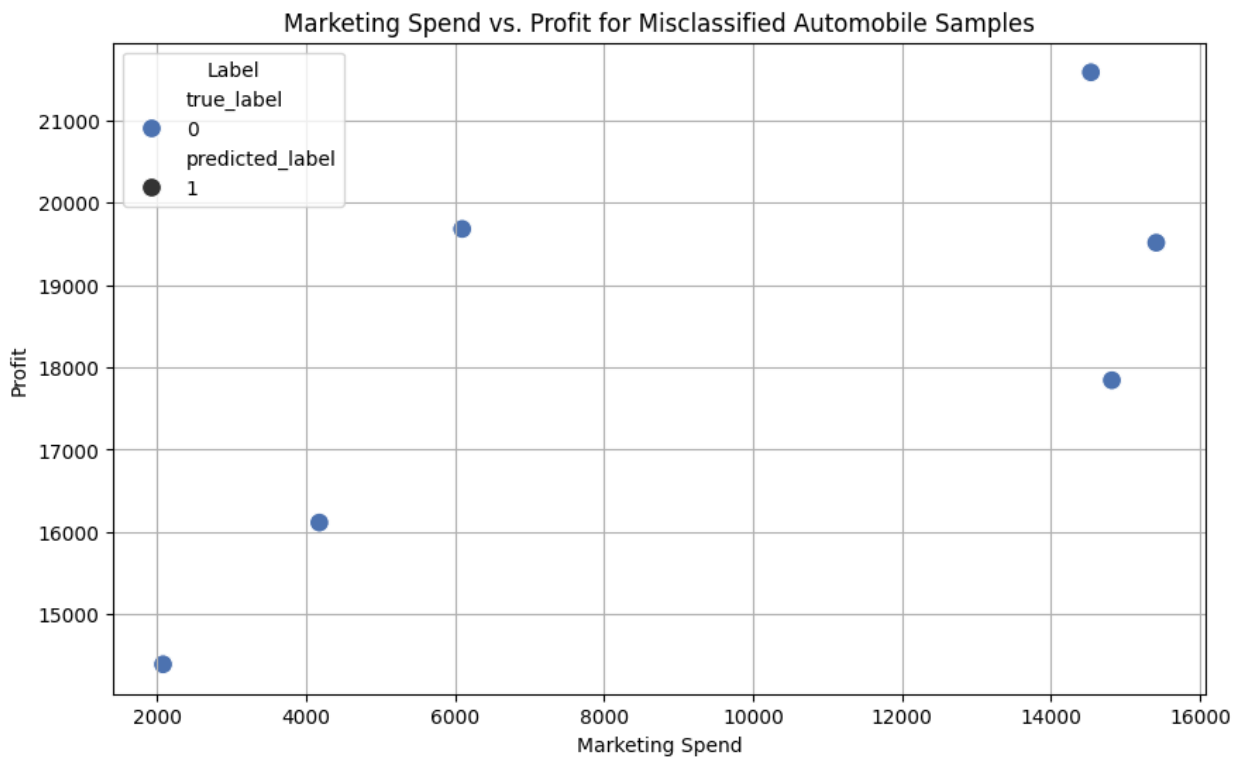


Visualization of Misclassified 'Automobile' Samples

To further investigate the misclassifications in the 'Automobile' category, let's visualize 'Marketing_Spend' vs. 'Profit' specifically for these samples. This might reveal certain thresholds or ranges where the model struggles.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Filter for misclassified 'Automobile' samples
misclassified_automobile_df =
automobile_df[automobile_df['true_label'] !=
automobile_df['predicted_label']]

if not misclassified_automobile_df.empty:
    plt.figure(figsize=(10, 6))
    sns.scatterplot(
        x='Marketing_Spend',
        y='Profit',
        hue='true_label', # Show true label for context if desired
        style='predicted_label', # Show predicted label as style for
contrast
        data=misclassified_automobile_df,
```

```
        s=100, # Size of points
        palette='deep'
    )
    plt.title('Marketing Spend vs. Profit for Misclassified Automobile
Samples')
    plt.xlabel('Marketing Spend')
    plt.ylabel('Profit')
    plt.grid(True)
    plt.legend(title='Label')
    plt.show()
else:
    print("No misclassified 'Automobile' samples to visualize.")
```



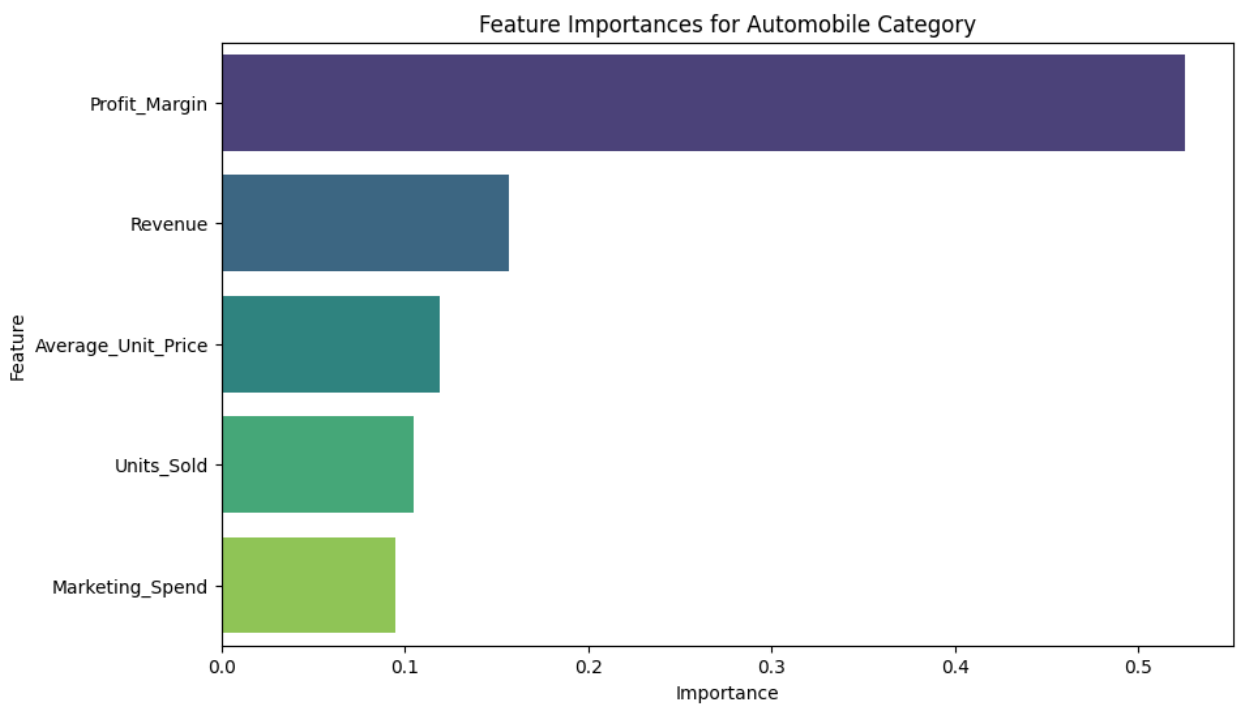
Feature Importance for 'Automobile' Product Category

To understand which features are most influential for predictions within the 'Automobile' category, we will train a specific model for this segment and analyze its feature importances.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Filter data for 'Automobile' category
df_automobile = df[df['Product_Category'] == 'Automobile'].copy()
```

```
# Re-create Profit Margin and Average Unit Price for df_automobile
df_automobile['Profit_Margin'] = df_automobile['Profit'] /
df_automobile['Revenue']
df_automobile['Average_Unit_Price'] = df_automobile.apply(lambda row:
row['Revenue'] / row['Units_Sold'] if row['Units_Sold'] > 0 else 0,
axis=1)

# 2. Prepare features and target for 'Automobile' data
# Assuming 'Profit' is used to derive target and has been removed from
features globally
X_auto = df_automobile[['Revenue', 'Units_Sold', 'Marketing_Spend',
'Profit_Margin', 'Average_Unit_Price']]
y_auto = (df_automobile['Profit'] >
df_automobile['Profit'].mean()).astype(int)

# Split data (ensure enough samples for train/test split if category
is small)
if len(X_auto) > 1:
    # Ensure stratify is used if y_auto has more than one class.
    if len(y_auto.unique()) > 1:
        X_train_auto, X_test_auto, y_train_auto, y_test_auto =
train_test_split(X_auto, y_auto, test_size=0.2, random_state=42,
stratify=y_auto)
    else:
        print("Target variable 'y_auto' has only one class for
'Automobile' category, skipping stratification.")
        X_train_auto, X_test_auto, y_train_auto, y_test_auto =
train_test_split(X_auto, y_auto, test_size=0.2, random_state=42)
else:
    print("Not enough samples in 'Automobile' category for train/test
split. Skipping model training.")
    X_train_auto, y_train_auto = X_auto, y_auto # Use all for training
if only few samples

# 3. Train a new RandomForestClassifier for 'Automobile' data
# Only proceed if X_train_auto is not empty (i.e., if there were
enough samples for a split or direct assignment)
if not X_train_auto.empty:
    model_automobile = RandomForestClassifier(n_estimators=100,
random_state=42)
    model_automobile.fit(X_train_auto, y_train_auto)

    # 4. Extract and visualize feature importances
    feature_importances_auto = model_automobile.feature_importances_
    feature_names_auto = X_auto.columns

    importance_df_auto = pd.DataFrame({
        'Feature': feature_names_auto,
        'Importance': feature_importances_auto
```

```
    })

    importance_df_auto =
importance_df_auto.sort_values(by='Importance', ascending=False)

    plt.figure(figsize=(10, 6))
    sns.barplot(x='Importance', y='Feature', data=importance_df_auto,
palette='viridis', hue='Feature', legend=False)
    plt.title('Feature Importances for Automobile Category')
    plt.xlabel('Importance')
    plt.ylabel('Feature')
    plt.show()
else:
    print("Skipping feature importance visualization as no model was
trained due to insufficient data.")
```




```
total_revenue = df['Revenue'].sum()
# avg_deal_size = df['deal_size'].mean() # 'deal_size' column does not
exist

# Drop 'Marketing_Efficiency' from X_train and X_test
X_train = X_train.drop(columns=['Marketing_Efficiency'])
X_test = X_test.drop(columns=['Marketing_Efficiency'])

print("Dropped 'Marketing_Efficiency' from X_train. New X_train
features:")
display(X_train.head())
```

```
print("Dropped 'Marketing_Efficiency' from X_test. New X_test
features:")
display(X_test.head())
```

```
Dropped 'Marketing_Efficiency' from X_train. New X_train features:
```

{"summary":"{\n \"name\": \"display(X_test\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"Revenue\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 57126,\n \"min\": 51321,\n \"max\": 196321,\n \"num_unique_values\": 5,\n \"samples\": [\n 196321,\n 51321,\n 177296\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Units_Sold\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 390,\n \"min\": 31,\n \"max\": 949,\n \"num_unique_values\": 5,\n \"samples\": [\n 31,\n 877,\n 910\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Marketing_Spend\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 4318,\n \"min\": 8496,\n \"max\": 19147,\n \"num_unique_values\": 5,\n \"samples\": [\n 19147,\n 8496,\n 12023\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Profit_Margin\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.057731474531001414,\n \"min\": 0.06306508218682667,\n \"max\": 0.20700415210896755,\n \"num_unique_values\": 5,\n \"samples\": [\n 0.06306508218682667,\n 0.18715535550749207,\n 0.17853758686039167\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Average_Unit_Price\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 2750.983865070728,\n \"min\": 58.5188141391106,\n \"max\": 6332.935483870968,\n \"num_unique_values\": 5,\n \"samples\": [\n 6332.935483870968,\n 58.5188141391106,\n 194.83076923076922\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

```
Dropped 'Marketing_Efficiency' from X_test. New X_test features:
```

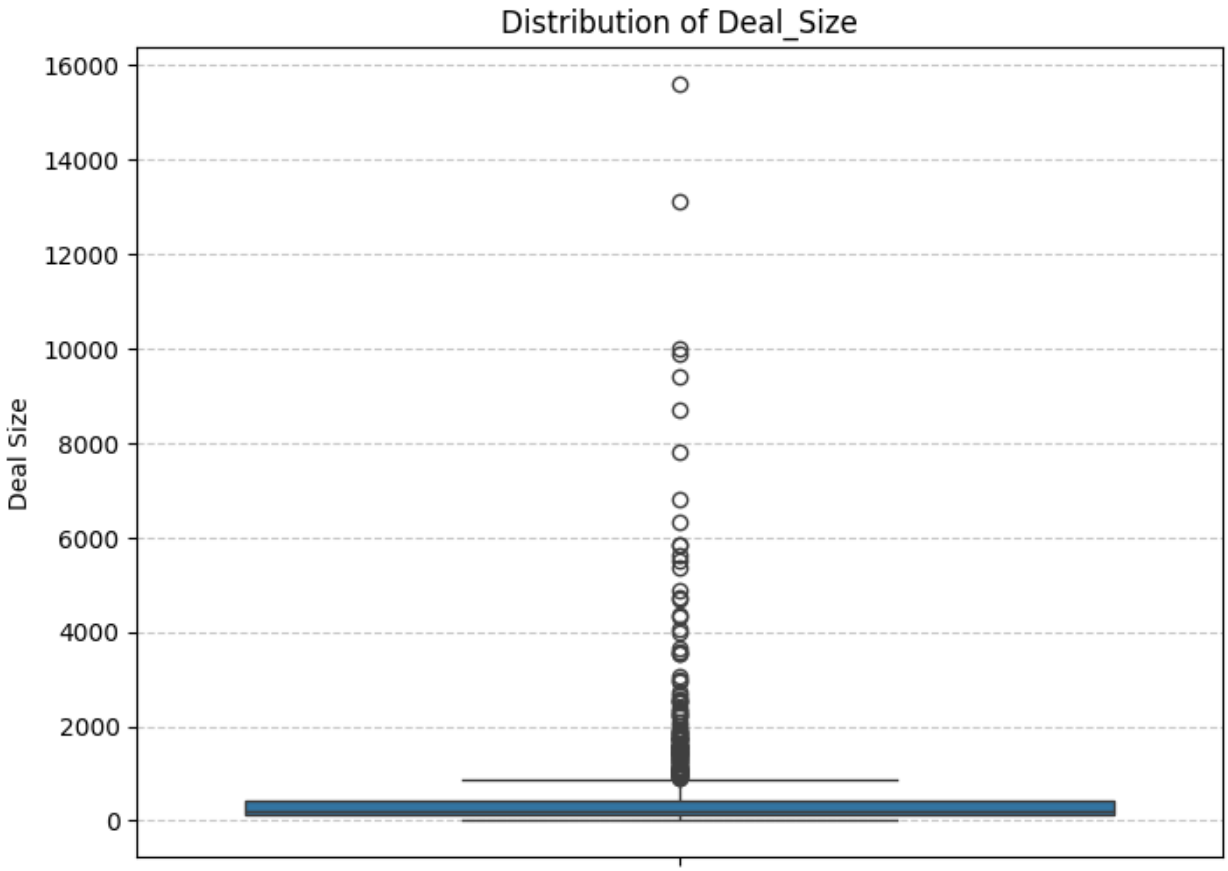
{"summary":"{\n \"name\": \"display(X_test\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"Revenue\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 46845,\n \"min\": 73133,\n \"max\": 198785,\n \"num_unique_values\": 5,\n \"samples\": [\n 133257,\n 119847,\n 198785\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Units_Sold\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 214,\n \"min\": 121,\n

\"max\": 660,\n   \"num_unique_values\": 5,\n   \"samples\": [\n   413,\n   660,\n   347\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Marketing_Spend\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 6965,\n   \"min\": 1629,\n   \"max\": 17076,\n   \"num_unique_values\": 5,\n   \"samples\": [\n   17076,\n   8988,\n   2910\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Profit_Margin\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.09903918441844127,\n   \"min\": 0.011684189198316037,\n   \"max\": 0.24222990989020005,\n   \"num_unique_values\": 5,\n   \"samples\": [\n   0.011684189198316037,\n   0.19115205219988818,\n   0.07492013984958623\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Average_Unit_Price\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 188.62242610875694,\n   \"min\": 181.58636363636364,\n   \"max\": 604.404958677686,\n   \"num_unique_values\": 5,\n   \"samples\": [\n   322.6561743341404,\n   181.58636363636364,\n   572.8674351585014\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

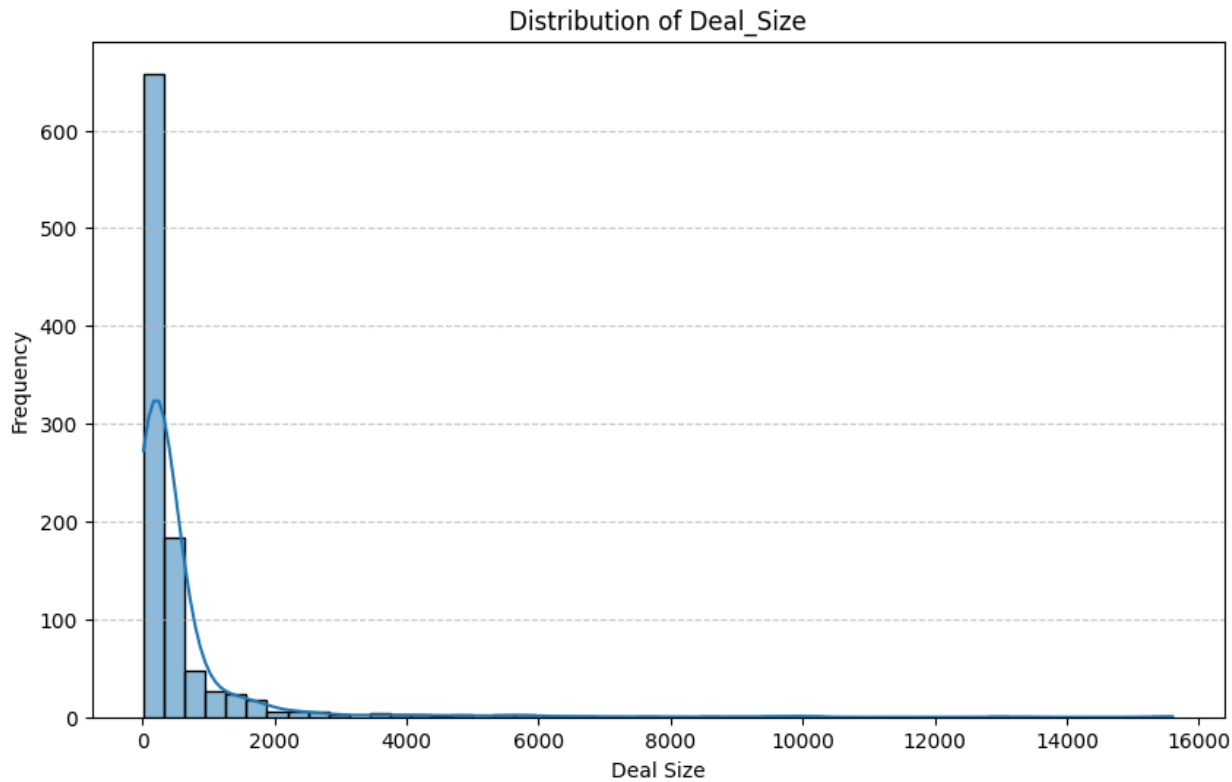
```python
# Calculate 'Deal_Size' (Revenue / Units_Sold), handling division by zero
df['Deal_Size'] = df.apply(lambda row: row['Revenue'] / row['Units_Sold'] if row['Units_Sold'] > 0 else 0, axis=1)

# Calculate the average deal size
average_deal_size = df['Deal_Size'].mean()

print(f"Average Deal Size: {average_deal_size:.2f}")
```

```
Average Deal Size: 515.98
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.boxplot(y=df['Deal_Size'])
plt.title('Distribution of Deal_Size')
plt.ylabel('Deal Size')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```




```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.histplot(df['Deal_Size'], bins=50, kde=True)
plt.title('Distribution of Deal_Size')
plt.xlabel('Deal Size')
plt.ylabel('Frequency')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```




```python
# Calculate Q1 (25th percentile) and Q3 (75th percentile)
Q1 = df['Deal_Size'].quantile(0.25)
Q3 = df['Deal_Size'].quantile(0.75)

# Calculate IQR (Interquartile Range)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers = df[(df['Deal_Size'] < lower_bound) | (df['Deal_Size'] >
upper_bound)]

print(f"Q1 (25th percentile): {Q1:.2f}")
print(f"Q3 (75th percentile): {Q3:.2f}")
print(f"IQR: {IQR:.2f}")
print(f"Lower Bound for Outliers: {lower_bound:.2f}")
print(f"Upper Bound for Outliers: {upper_bound:.2f}")

print(f"\nNumber of identified outliers: {len(outliers)}")
if not outliers.empty:
    print("\nPotential outliers in 'Deal_Size':")
    display(outliers[['Revenue', 'Units_Sold', 'Deal_Size']].head())
```

```
else:
    print("No outliers identified based on the IQR method.")
```

```
Q1 (25th percentile): 117.06
Q3 (75th percentile): 425.07
IQR: 308.01
Lower Bound for Outliers: -344.95
Upper Bound for Outliers: 887.08

Number of identified outliers: 115

Potential outliers in 'Deal_Size':
```


```
{"summary":"{\n  \"name\": \"    print(\\\"No outliers identified based on the IQR method\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n  \"column\": \"Revenue\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 47207,\n        \"min\": 72768,\n        \"max\": 187235,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          72768,\n          105081,\n          187235\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Units_Sold\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 37,\n        \"min\": 12,\n        \"max\": 109,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          40,\n          74,\n          12\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Deal_Size\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 6146.856914455098,\n        \"min\": 1420.0135135135135,\n        \"max\": 15602.916666666666,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          1819.2,\n          1420.0135135135135,\n          15602.916666666666\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Investigating High 'Deal_Size' Outliers

The 'Deal_Size' is calculated as `Revenue / Units_Sold`. Therefore, unusually high 'Deal_Size' values can typically be attributed to two main factors:

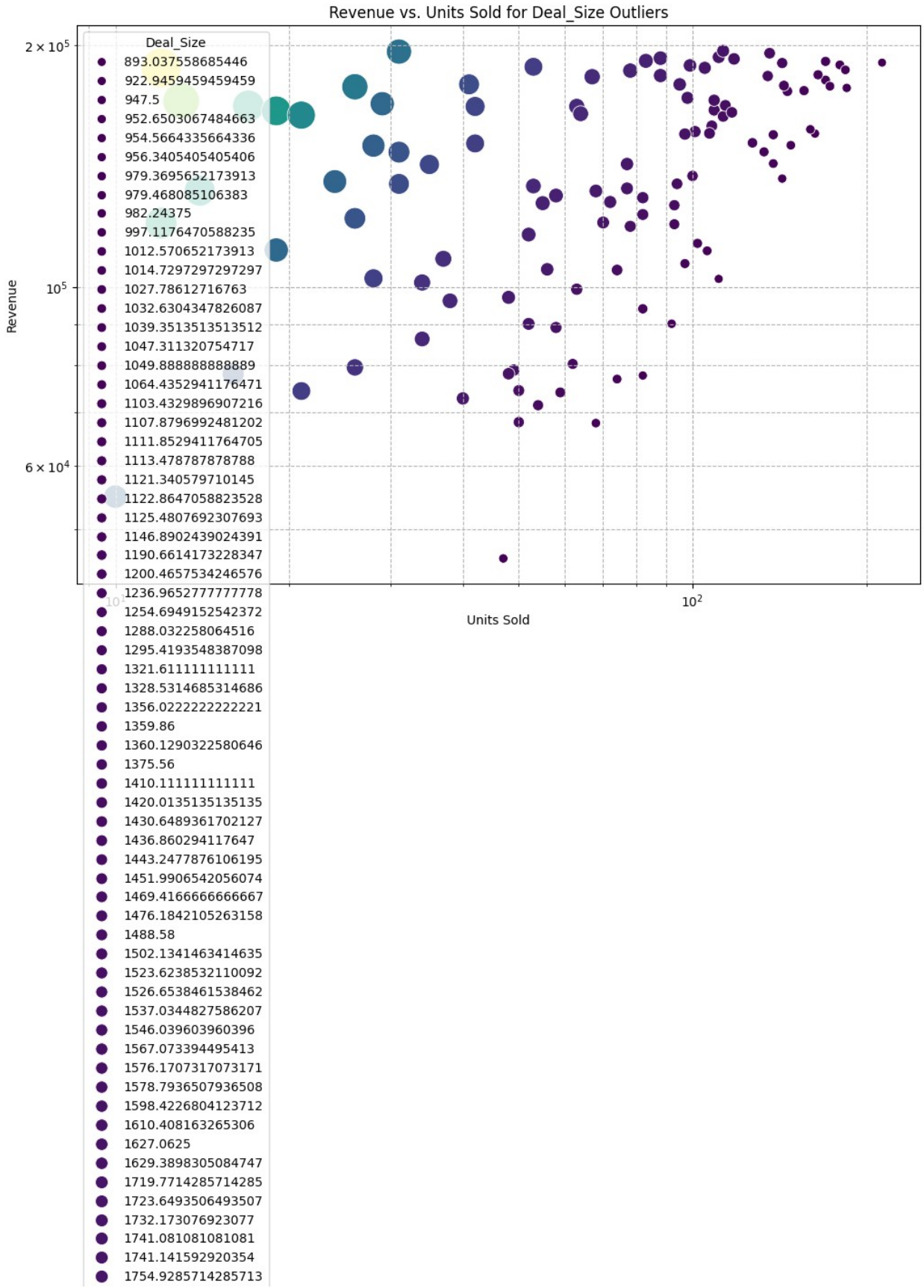
1. **Very High Revenue:** The transaction involved a significantly large amount of money.
2. **Very Low Units Sold:** The transaction involved a small number of units, making the revenue per unit (deal size) disproportionately high.

```
pipeline_value = df[df['is_closed'] == 0]['deal_size'].sum()

print("Top 10 Outliers by 'Deal_Size':")
display(outliers.sort_values(by='Deal_Size', ascending=False).head(10)[['Revenue', 'Units_Sold', 'Deal_Size']])
```

```
Top 10 Outliers by 'Deal_Size':
```

{"summary":"{\n \"name\": \"display(outliers\",\n \"rows\": 10,\n \"fields\": [\n {\n \"column\": \"Revenue\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 28546,\n \"min\": 111268,\n \"max\": 196321,\n \"num_unique_values\": 10,\n \"samples\": [\n 196321,\n 170400,\n 165683\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Units_Sold\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 6,\n \"min\": 12,\n \"max\": 31,\n \"num_unique_values\": 8,\n \"samples\": [\n 13,\n 21,\n 12\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Deal_Size\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 3063.0131814340157,\n \"min\": 5856.210526315789,\n \"max\": 15602.916666666666,\n \"num_unique_values\": 10,\n \"samples\": [\n 6332.935483870968,\n 13107.692307692309,\n 8720.157894736842\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 8))
sns.scatterplot(
    x='Units_Sold',
    y='Revenue',
    size='Deal_Size', # Size points by Deal_Size
    hue='Deal_Size',  # Color points by Deal_Size
    data=outliers,
    sizes=(50, 1000), # Range of point sizes
    palette='viridis', # Color palette
    legend='full'
)
plt.title('Revenue vs. Units Sold for Deal_Size Outliers')
plt.xlabel('Units Sold')
plt.ylabel('Revenue')
plt.xscale('log') # Use log scale for Units_Sold if it spans a wide range
plt.yscale('log') # Use log scale for Revenue if it spans a wide range
plt.grid(True, which="both", ls="--", c='0.7')
plt.show()
```


Revenue vs. Units Sold for Deal_Size Outliers
Deal_Size
893.037558685446
922.9459459459459
947.5
952.6503067484663
954.5664335664336
956.3405405405406
979.3695652173913
979.468085106383
982.24375
997.1176470588235
1012.570652173913
1014.7297297297297
1027.78612716763
1032.6304347826087
1039.3513513513512
1047.311320754717
1049.888888888889
1064.4352941176471
1103.4329896907216
1107.8796992481202
1111.8529411764705
1113.478787878788
1121.340579710145
1122.8647058823528
1125.4807692307693
1146.8902439024391
1190.6614173228347
1200.4657534246576
1236.9652777777778
1254.6949152542372
1288.032258064516
1295.4193548387098
1321.611111111111
1328.5314685314686
1356.0222222222221
1359.86
1360.1290322580646
1375.56
1410.111111111111
1420.0135135135135
1430.6489361702127
1436.860294117647
1443.2477876106195
1451.9906542056074
1469.4166666666667
1476.1842105263158
1488.58
1502.1341463414635
1523.6238532110092
1526.6538461538462
1537.0344827586207
1546.039603960396
1567.073394495413
1576.1707317073171
1578.7936507936508
1598.4226804123712
1610.408163265306
1627.0625
1629.3898305084747
1719.7714285714285
1723.6493506493507
1732.173076923077
1741.081081081081
1741.141592920354
1754.9285714285713
Revenue
2×10^5
10^5
6×10^4
10^2
Units Sold

Model Performance Comparison: With vs. Without 'Deal_Size' Outliers

To assess the impact of the 'Deal_Size' outliers, we will create a new dataset by removing these identified outliers. We will then retrain our previously tuned Random Forest model on this cleaner data and compare its performance metrics against the model trained with outliers.

```
# 1. Create a DataFrame without outliers
df_no_outliers = df.drop(outliers.index)

# Re-create Profit Margin and Average Unit Price for df_no_outliers
df_no_outliers['Profit_Margin'] = df_no_outliers['Profit'] /
df_no_outliers['Revenue']
df_no_outliers['Average_Unit_Price'] = df_no_outliers.apply(lambda
row: row['Revenue'] / row['Units_Sold'] if row['Units_Sold'] > 0 else
0, axis=1)

# 2. Redefine features (X) and target (y) for the outlier-free dataset
# Ensure 'Marketing_Efficiency' is not included as it was previously
dropped due to multicollinearity
X_no_outliers = df_no_outliers[['Revenue', 'Units_Sold',
'Marketing_Spend', 'Profit_Margin', 'Average_Unit_Price']]
y_no_outliers = (df_no_outliers['Profit'] >
df_no_outliers['Profit'].mean()).astype(int)

print(f"Original dataset shape: {df.shape}")
print(f"Dataset shape after removing {len(outliers)} outliers:
{df_no_outliers.shape}")

# 3. Re-split the data
from sklearn.model_selection import train_test_split
X_train_no_outliers, X_test_no_outliers, y_train_no_outliers,
y_test_no_outliers = train_test_split(
    X_no_outliers, y_no_outliers, test_size=0.2, random_state=42
)

print("Data re-split after outlier removal.")
print(f"X_train_no_outliers shape: {X_train_no_outliers.shape}")
print(f"X_test_no_outliers shape: {X_test_no_outliers.shape}")
```

```
Original dataset shape: (1000, 12)
Dataset shape after removing 115 outliers: (885, 14)
Data re-split after outlier removal.
X_train_no_outliers shape: (708, 5)
X_test_no_outliers shape: (177, 5)
```

```
# 4. Retrain the model with the best hyperparameters on the outlier-
free data
from sklearn.ensemble import RandomForestClassifier

# Assuming 'best_params' from previous GridSearchCV is available
```

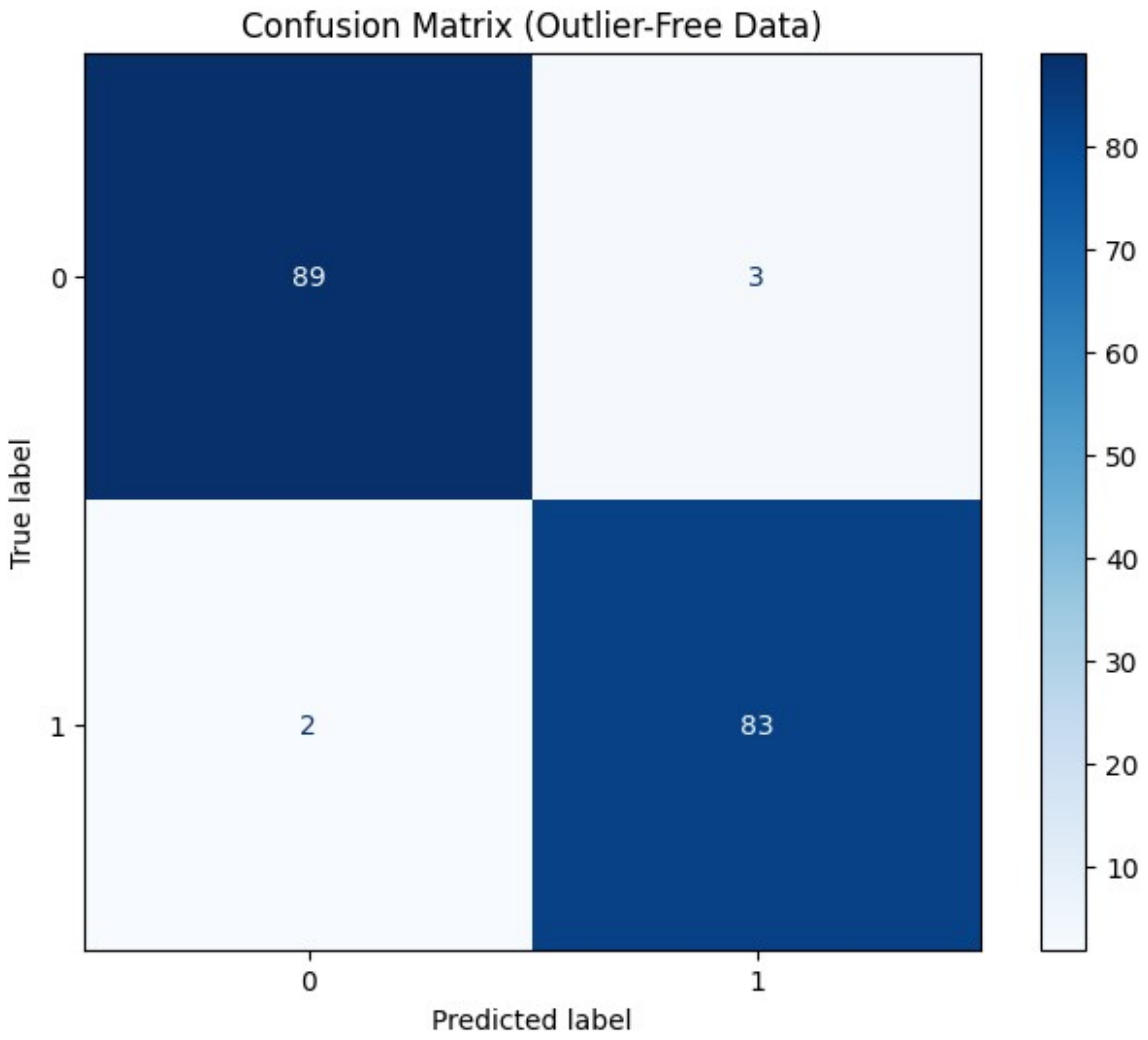
```python
# If not, you might need to re-run GridSearchCV or define best_params manually

best_model_no_outliers = RandomForestClassifier(random_state=42, **best_params)
best_model_no_outliers.fit(X_train_no_outliers, y_train_no_outliers)

print("Model retrained with best hyperparameters on outlier-free data.")

# Make predictions
y_pred_no_outliers = best_model_no_outliers.predict(X_test_no_outliers)
y_pred_proba_no_outliers = best_model_no_outliers.predict_proba(X_test_no_outliers)[:, 1]
```

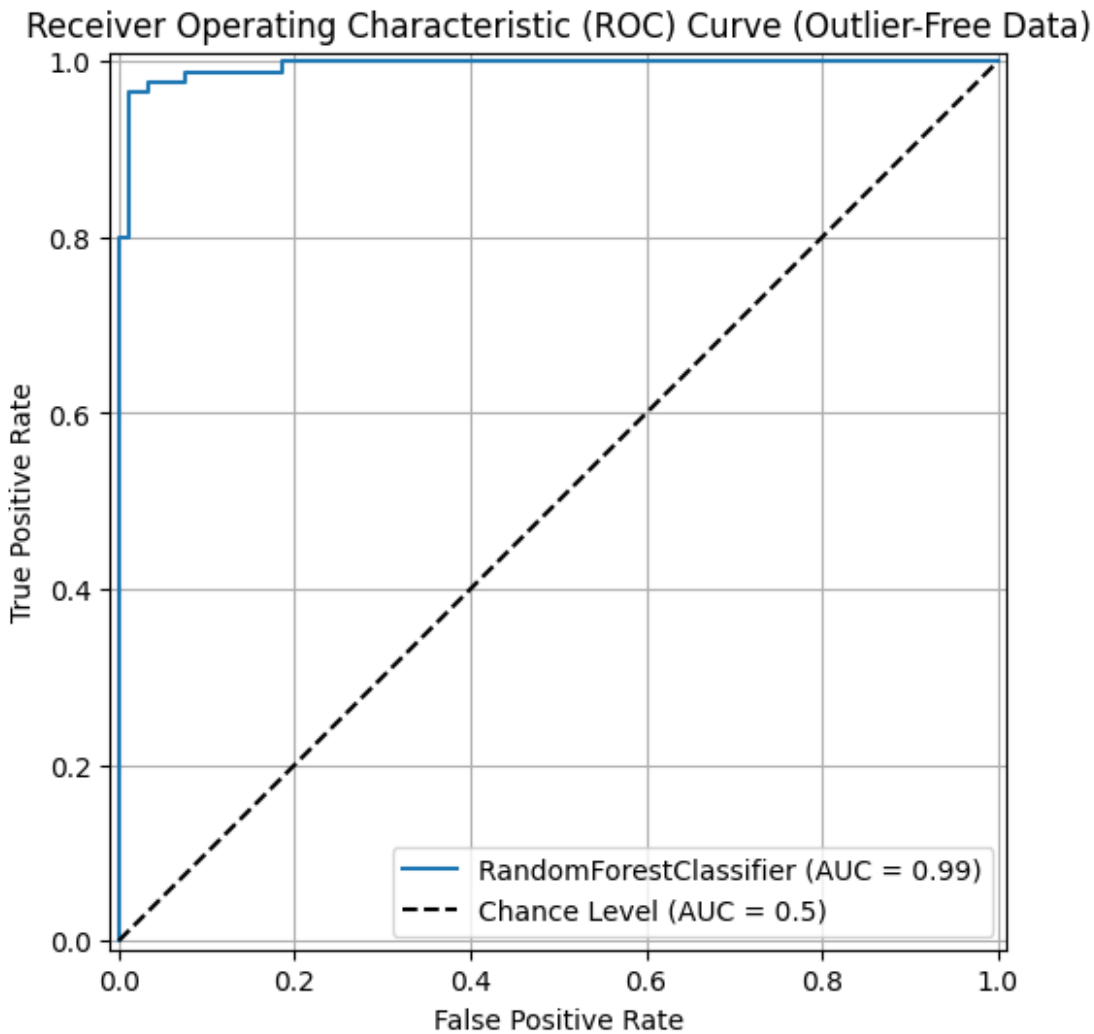
```
Model retrained with best hyperparameters on outlier-free data.
```

```python
# 5. Evaluate the retrained model (outlier-free)
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report, roc_auc_score, RocCurveDisplay
import matplotlib.pyplot as plt

accuracy_no_outliers = accuracy_score(y_test_no_outliers, y_pred_no_outliers)
precision_no_outliers = precision_score(y_test_no_outliers, y_pred_no_outliers)
recall_no_outliers = recall_score(y_test_no_outliers, y_pred_no_outliers)
f1_no_outliers = f1_score(y_test_no_outliers, y_pred_no_outliers)
auc_no_outliers = roc_auc_score(y_test_no_outliers, y_pred_proba_no_outliers)

print("\n--- Performance Metrics (Outlier-Free Data) ---")
print(f"Accuracy: {accuracy_no_outliers:.4f}")
print(f"Precision: {precision_no_outliers:.4f}")
print(f"Recall: {recall_no_outliers:.4f}")
print(f"F1-Score: {f1_no_outliers:.4f}")
print(f"AUC Score: {auc_no_outliers:.4f}")

print("\nClassification Report (Outlier-Free Data):")
print(classification_report(y_test_no_outliers, y_pred_no_outliers))

# Confusion Matrix
cm_no_outliers = confusion_matrix(y_test_no_outliers, y_pred_no_outliers)
fig, ax = plt.subplots(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_no_outliers, display_labels=best_model_no_outliers.classes_)
```

```
disp.plot(cmap=plt.cm.Blues, ax=ax)
plt.title('Confusion Matrix (Outlier-Free Data)')
plt.show()

# ROC Curve
fig, ax = plt.subplots(figsize=(8, 6))
roc_display = RocCurveDisplay.from_estimator(best_model_no_outliers,
X_test_no_outliers, y_test_no_outliers, ax=ax)
plt.plot([0, 1], [0, 1], 'k--', label='Chance Level (AUC = 0.5)')
plt.title('Receiver Operating Characteristic (ROC) Curve (Outlier-Free
Data)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```

```
--- Performance Metrics (Outlier-Free Data) ---
Accuracy: 0.9718
Precision: 0.9651
Recall: 0.9765
F1-Score: 0.9708
AUC Score: 0.9948

Classification Report (Outlier-Free Data):
              precision    recall  f1-score   support

           0       0.98      0.97      0.97        92
           1       0.97      0.98      0.97        85

    accuracy                           0.97       177
   macro avg       0.97      0.97      0.97       177
weighted avg       0.97      0.97      0.97       177
```


Confusion Matrix (Outlier-Free Data)
True label
Predicted label
0
1
0
1
89
3
2
83
80
70
60
50
40
30
20
10



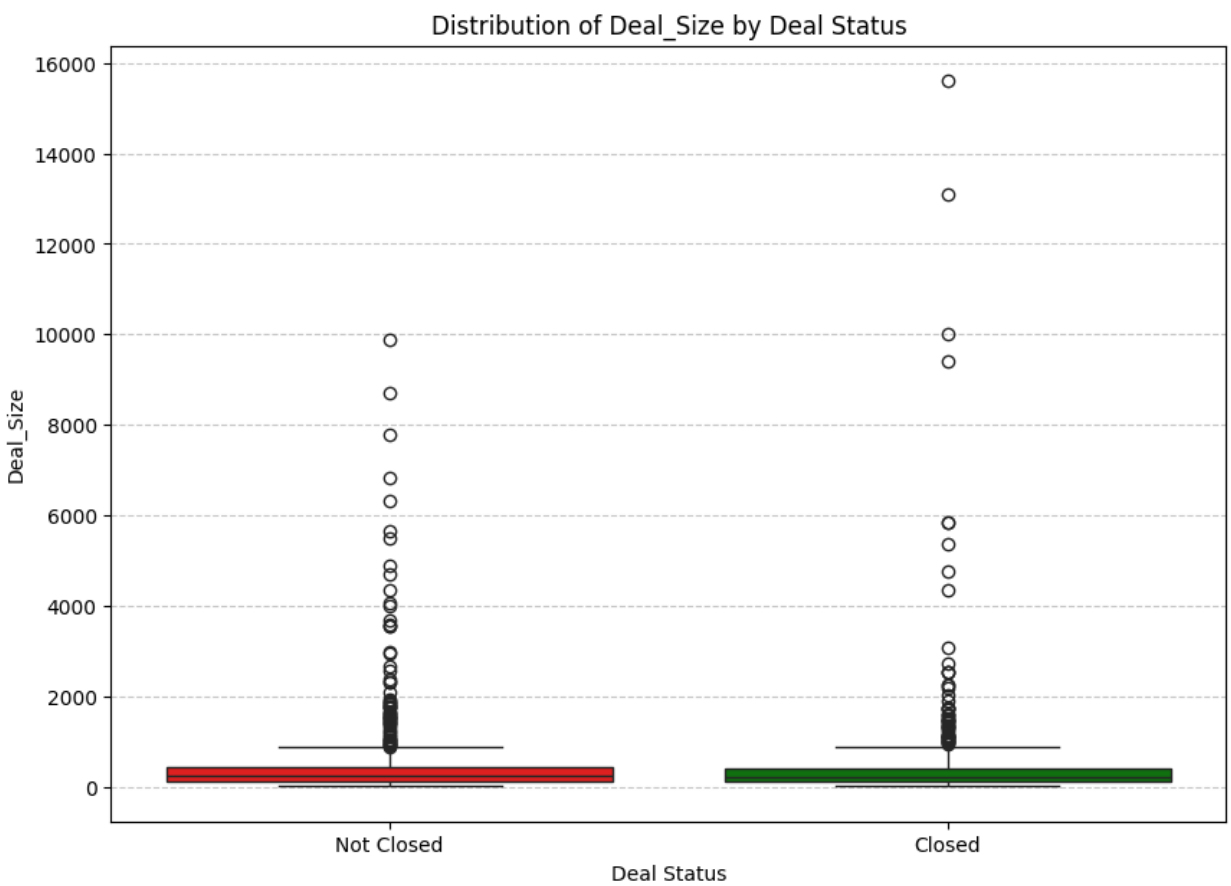

```
pipeline_value = df[df['is_closed'] == 0]['deal_size'].sum()

pipeline_value = df[y == 0]['Deal_Size'].sum() # Assuming y=0
represents 'not closed'
```

Distribution of 'Deal_Size' for Closed vs. Not Closed Deals

Let's visualize the distribution of the `Deal_Size` for deals that were 'closed' (our target `y=1`) versus those that were 'not closed' (`y=0`). This can provide insights into whether deal size is a significant differentiator between successful and unsuccessful deals.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a temporary DataFrame for plotting, aligning 'Deal_Size' with
the target 'y'
# Make sure to reset the index of y to align correctly with df if df
```

```
has been reindexed or filtered
plot_df = pd.DataFrame({
    'Deal_Size': df['Deal_Size'],
    'Deal_Status': y.map({0: 'Not Closed', 1: 'Closed'}) # Map
numerical target to descriptive labels
}).dropna()

plt.figure(figsize=(10, 7))
sns.boxplot(x='Deal_Status', y='Deal_Size', data=plot_df,
hue='Deal_Status', palette={'Closed': 'green', 'Not Closed': 'red'},
legend=False)
plt.title('Distribution of Deal_Size by Deal Status')
plt.xlabel('Deal Status')
plt.ylabel('Deal_Size')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Mean and Median 'Deal_Size' Comparison

Let's compare the mean and median `Deal_Size` for 'Closed' and 'Not Closed' deals to get a statistical summary of their central tendencies.

```
print(df.columns)
```

```
Index(['Region', 'Salesperson', 'Revenue', 'Profit', 'Units_Sold',
       'Customer_Satisfaction', 'Marketing_Spend',
'Discount_Percentage',
       'Product_Category', 'Sales_Channel', 'Order_Processing_Time',
       'Deal_Size'],
      dtype='object')
```

```
avg_sales_cycle = df['Order_Processing_Time'].mean()
```

```
df.groupby('Region')['Revenue'].sum()
```

```
Region
east     25438195
north    29373063
south    23876202
west     28628148
Name: Revenue, dtype: int64
```

```
df['is_closed'] = y
df.groupby('Salesperson')['is_closed'].mean()
```

```
Salesperson
alice      0.516432
bob        0.410891
charlie    0.468293
david      0.526316
eve        0.464115
Name: is_closed, dtype: float64
```

```
total_revenue_by_salesperson = df.groupby('Salesperson')
['Revenue'].sum().reset_index()
total_revenue_by_salesperson.columns = ['Salesperson',
'Total_Revenue']

salesperson_performance = pd.merge(salesperson_win_prob,
total_revenue_by_salesperson, on='Salesperson')

print("Comparison of Win Probability and Total Revenue by
Salesperson:")
display(salesperson_performance.sort_values(by='Win_Probability',
ascending=False))
```

```
Comparison of Win Probability and Total Revenue by Salesperson:
```

{"summary":"{\n \"name\": \"display(salesperson_performance\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"Salesperson\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 5,\n \"samples\": [\n \"alice\",\n \"bob\",\n \"charlie\"\n

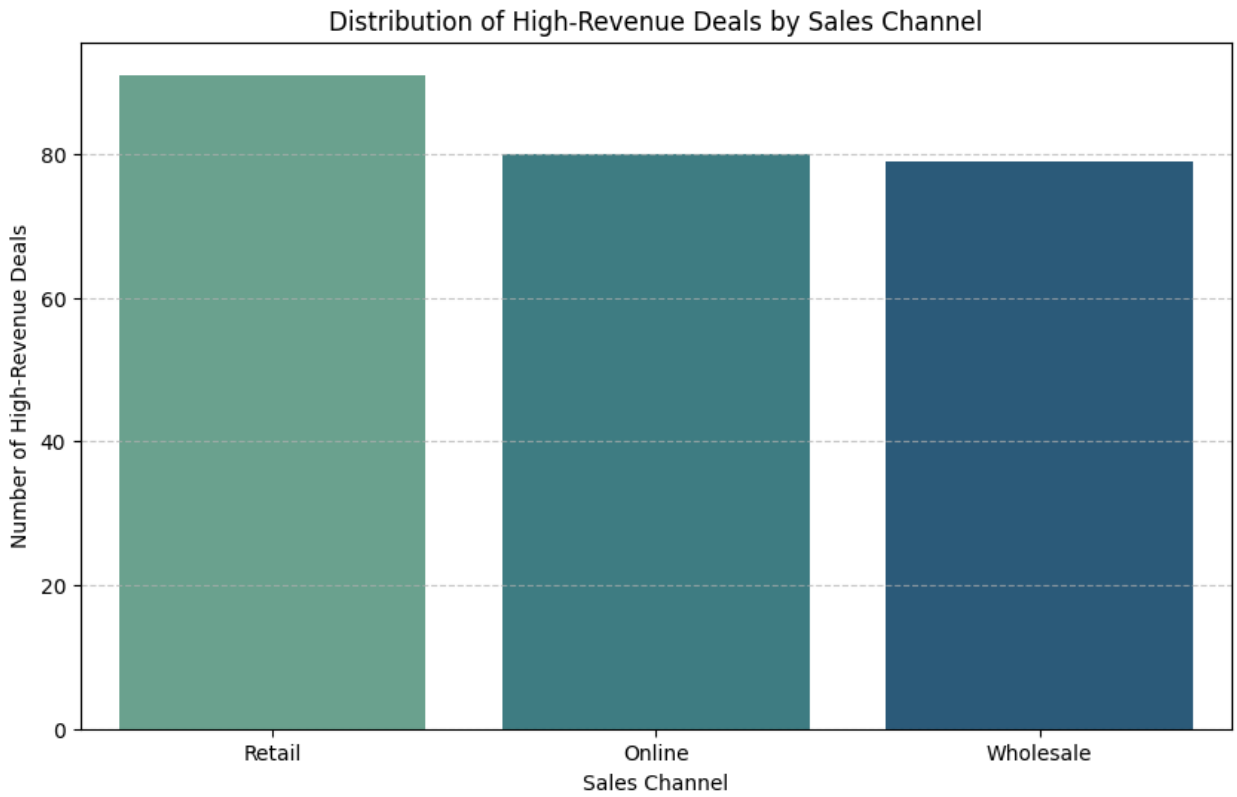
```
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Win_Probability\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.046365229011348666,\n        \"min\": 0.41089108910891087,\n
\"max\": 0.5263157894736842,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          0.5164319248826291,\n
0.41089108910891087,\n          0.4682926829268293\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Total_Revenue\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1957792,\n        \"min\": 18071268,\n        \"max\": 22945316,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
22945316,\n          22209168,\n          21599594\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```


```
# Calculate the 75th percentile of Revenue to define 'high-revenue'
deals
high_revenue_threshold = df['Revenue'].quantile(0.75)

# Filter the DataFrame to include only high-revenue deals
high_revenue_df = df[df['Revenue'] >= high_revenue_threshold]

print(f"High-revenue threshold (75th percentile): $
{high_revenue_threshold:,.2f}")
print(f"Number of high-revenue deals: {len(high_revenue_df)}")
```

```
High-revenue threshold (75th percentile): $155,577.75
Number of high-revenue deals: 250
```

Distribution of High-Revenue Deals by Sales Channel

```
import matplotlib.pyplot as plt
import seaborn as sns

# Analyze distribution by Sales_Channel
high_revenue_by_channel =
high_revenue_df['Sales_Channel'].value_counts().reset_index()
high_revenue_by_channel.columns = ['Sales_Channel', 'Count']

plt.figure(figsize=(10, 6))
sns.barplot(x='Sales_Channel', y='Count',
data=high_revenue_by_channel, palette='crest', hue='Sales_Channel',
legend=False)
plt.title('Distribution of High-Revenue Deals by Sales Channel')
plt.xlabel('Sales Channel')
plt.ylabel('Number of High-Revenue Deals')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Distribution of High-Revenue Deals by Region

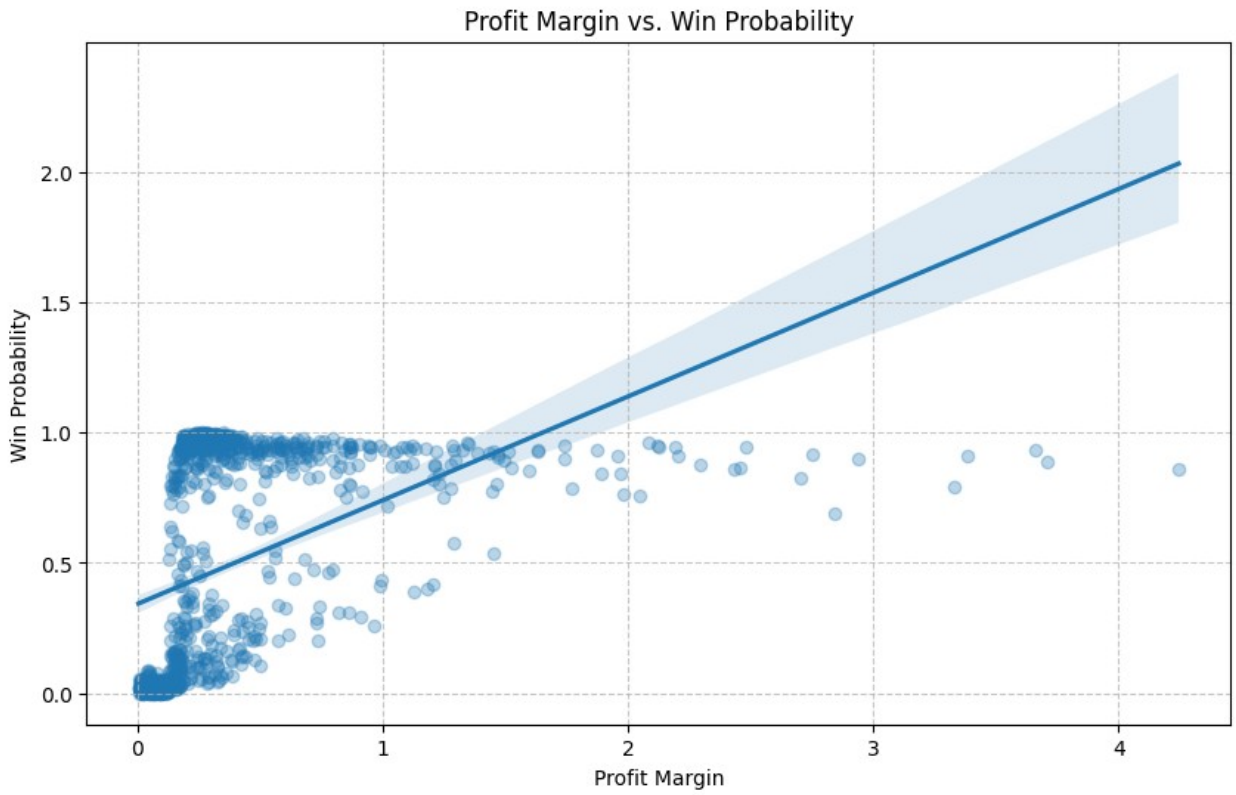
```
# Re-create Profit Margin and Average Unit Price (as df was reloaded
earlier)
df['Profit_Margin'] = df['Profit'] / df['Revenue']
df['Average_Unit_Price'] = df.apply(lambda row: row['Revenue'] /
row['Units_Sold'] if row['Units_Sold'] > 0 else 0, axis=1)

# Define the features for prediction, matching those used for
best_model training
X_for_prediction = df[['Revenue', 'Units_Sold', 'Marketing_Spend',
'Profit_Margin', 'Average_Unit_Price']]

# Calculate win_probability using the retrained best_model
df['win_probability'] = best_model.predict_proba(X_for_prediction)[:,
1]

df['expected_revenue'] = df['Deal_Size'] * df['win_probability']

print("Expected revenue calculated.")
display(df[['Deal_Size', 'win_probability',
'expected_revenue']].head())
```

```
Expected revenue calculated.
```

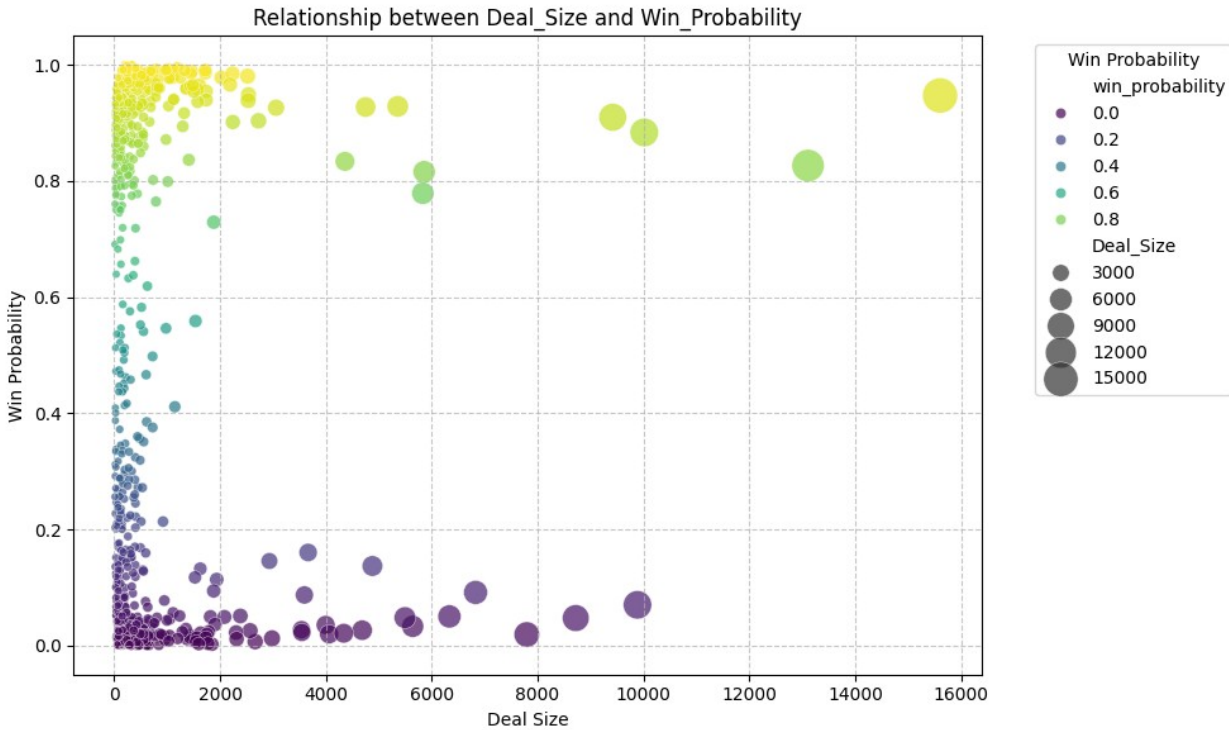
{"summary":"{\n \"name\": \"display(df[['Deal_Size', 'win_probability', 'expected_revenue']]\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"Deal_Size\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 812.4183277441518,\n \"min\": 46.99014778325123,\n \"max\": 1819.2,\n \"num_unique_values\": 5,\n \"samples\": [\n 104.76933333333334,\n 1819.2,\n 46.99014778325123\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"win_probability\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.4829848566205655,\n \"min\": 0.0017142857142857144,\n \"max\": 0.9035028123850276,\n \"num_unique_values\": 5,\n \"samples\": [\n 0.8975281629638565,\n 0.04954401154401155,\n 0.9035028123850276\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"expected_revenue\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 43.4079100633735,\n \"min\": 1.0733307240704502,\n \"max\": 94.0334272816146,\n \"num_unique_values\": 5,\n \"samples\": [\n 94.0334272816146,\n 90.13046580086582,\n 42.45573067655556\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate the correlation coefficient
correlation_profit_win =
df['Profit_Margin'].corr(df['win_probability'])
print(f"Correlation between Profit_Margin and Win_Probability:
{correlation_profit_win:.4f}")

# Create a scatter plot with a regression line
plt.figure(figsize=(10, 6))
sns.regplot(x='Profit_Margin', y='win_probability', data=df,
scatter_kws={'alpha':0.3})
plt.title('Profit Margin vs. Win Probability')
plt.xlabel('Profit Margin')
plt.ylabel('Win Probability')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

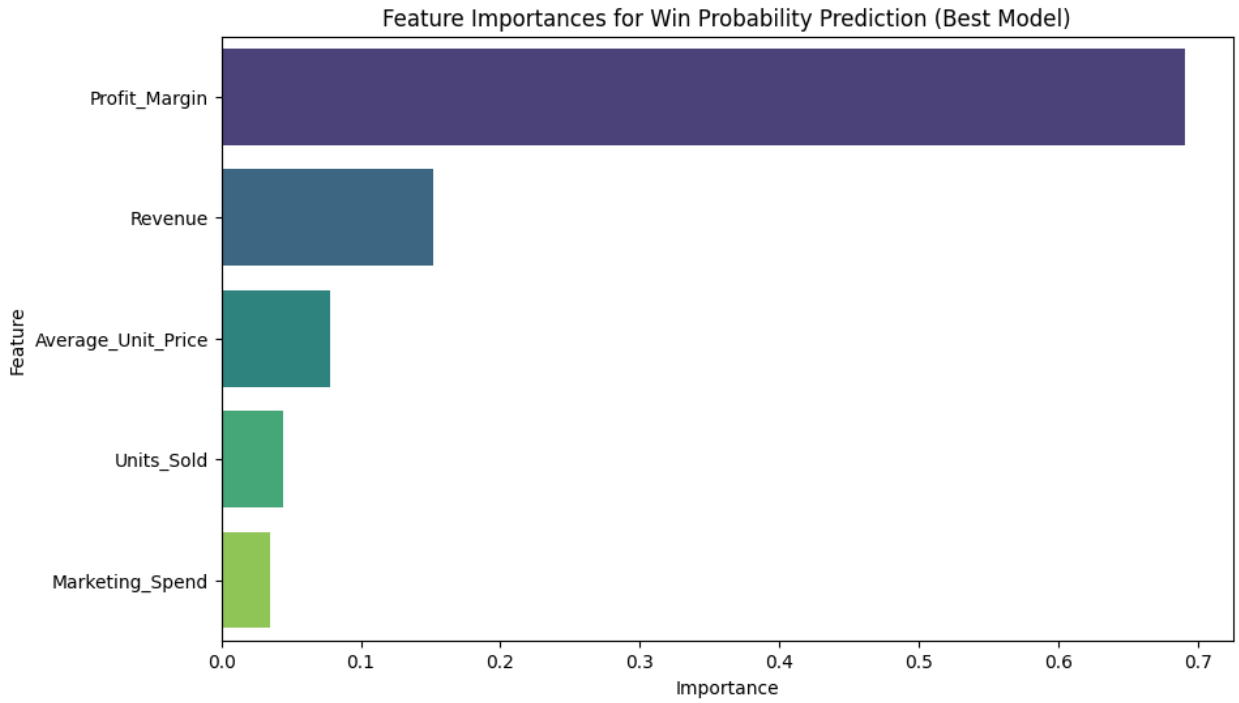
```
Correlation between Profit_Margin and Win_Probability: 0.4422
```



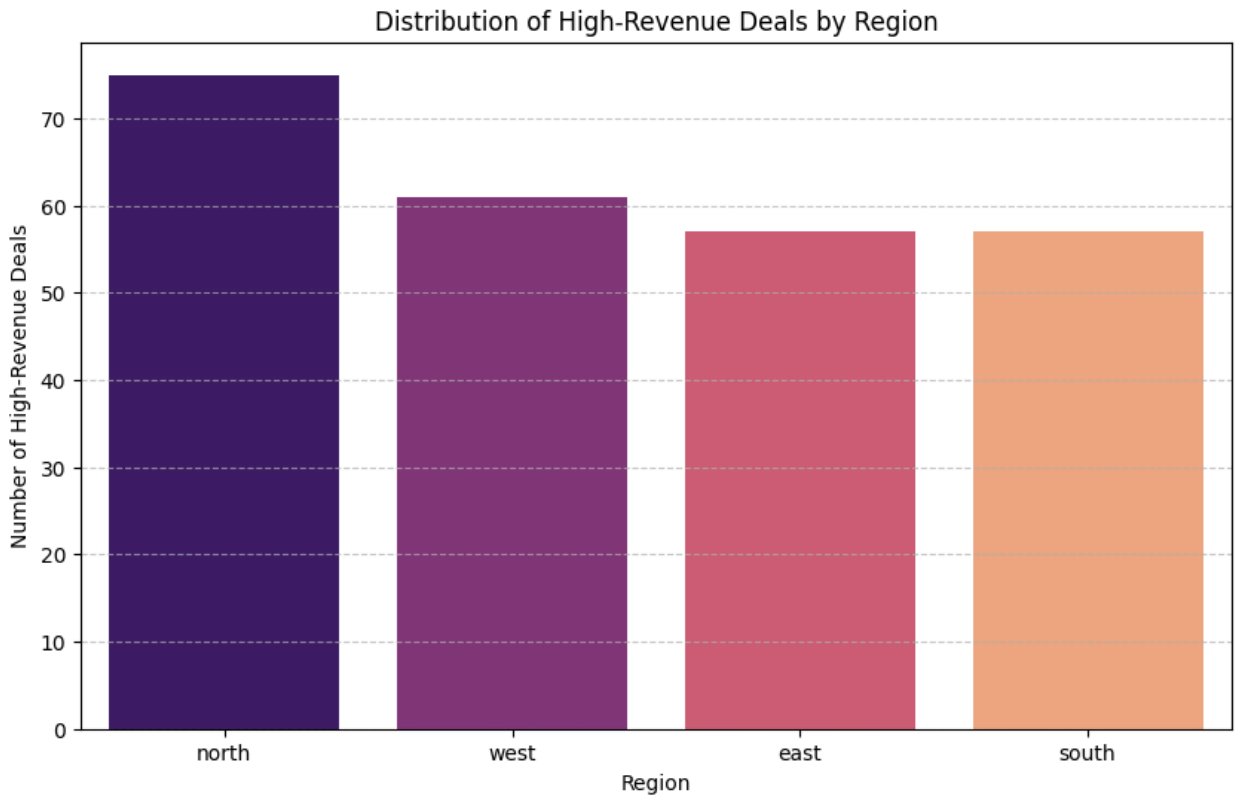

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.scatterplot(x='Deal_Size', y='win_probability', data=df,
hue='win_probability', palette='viridis', size='Deal_Size', sizes=(20,
400), alpha=0.7)
plt.title('Relationship between Deal_Size and Win_Probability')
plt.xlabel('Deal Size')
plt.ylabel('Win Probability')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(title='Win Probability', bbox_to_anchor=(1.05, 1),
loc='upper left')
plt.tight_layout()
plt.show()
```



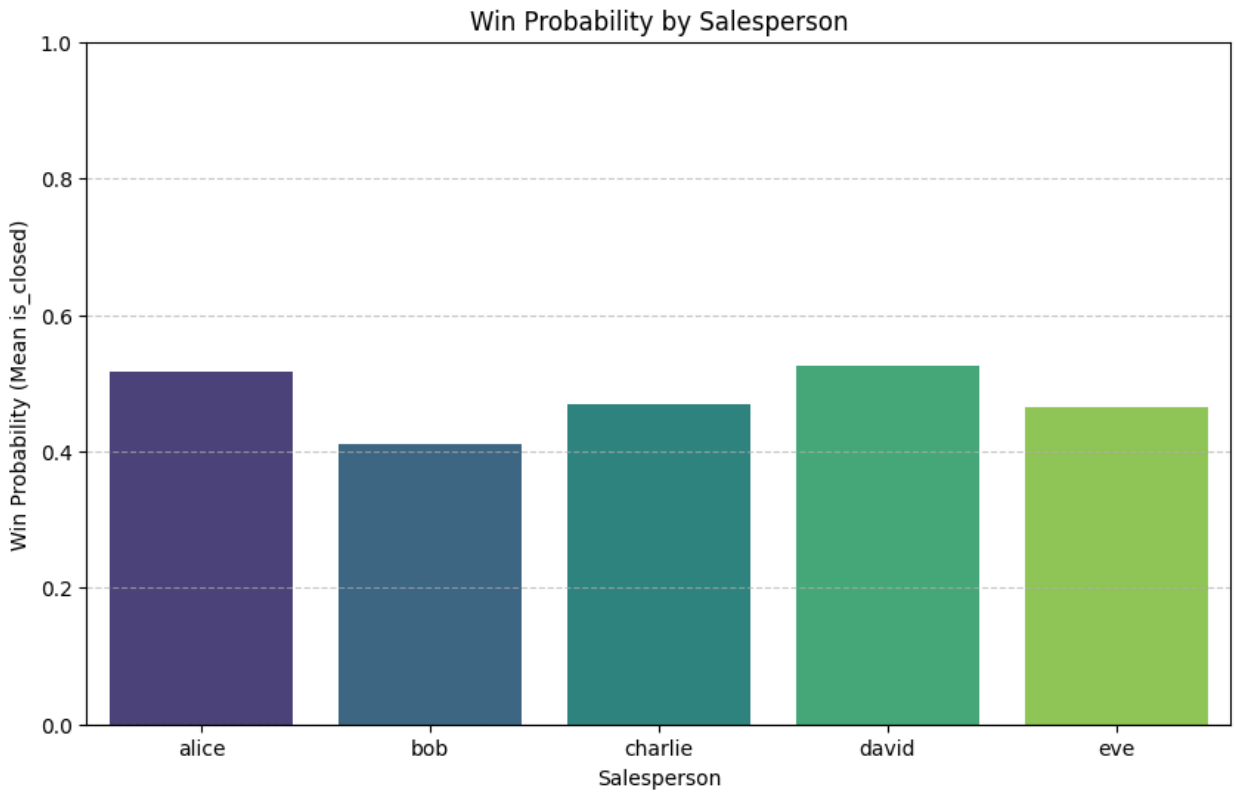

```
import matplotlib.pyplot as plt
import seaborn as sns

# Get feature importances from the best_model
feature_importances_best = best_model.feature_importances_
feature_names_best = X_for_prediction.columns

# Create a DataFrame for better visualization
importance_df_best = pd.DataFrame({
    'Feature': feature_names_best,
    'Importance': feature_importances_best
})

# Sort by importance
importance_df_best = importance_df_best.sort_values(by='Importance',
ascending=False)

# Plotting
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=importance_df_best,
palette='viridis', hue='Feature', legend=False)
plt.title('Feature Importances for Win Probability Prediction (Best
Model)')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.show()
```



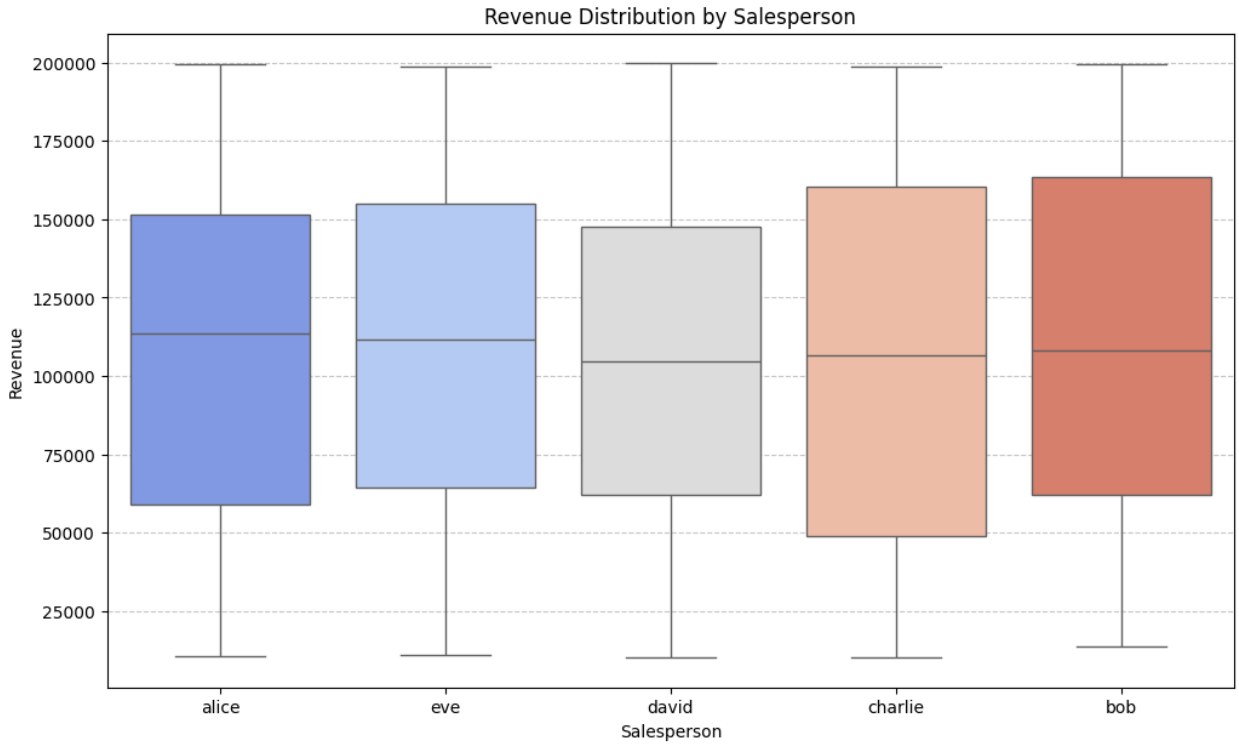

```
import matplotlib.pyplot as plt
import seaborn as sns

# Analyze distribution by Region
high_revenue_by_region =
high_revenue_df['Region'].value_counts().reset_index()
high_revenue_by_region.columns = ['Region', 'Count']

plt.figure(figsize=(10, 6))
sns.barplot(x='Region', y='Count', data=high_revenue_by_region,
palette='magma', hue='Region', legend=False)
plt.title('Distribution of High-Revenue Deals by Region')
plt.xlabel('Region')
plt.ylabel('Number of High-Revenue Deals')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```




```
import matplotlib.pyplot as plt
import seaborn as sns

salesperson_win_prob = df.groupby('Salesperson')
['is_closed'].mean().reset_index()
salesperson_win_prob.columns = ['Salesperson', 'Win_Probability']

plt.figure(figsize=(10, 6))
sns.barplot(x='Salesperson', y='Win_Probability',
data=salesperson_win_prob, palette='viridis', hue='Salesperson',
legend=False)
plt.title('Win Probability by Salesperson')
plt.xlabel('Salesperson')
plt.ylabel('Win Probability (Mean is_closed)')
plt.ylim(0, 1) # Probability ranges from 0 to 1
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

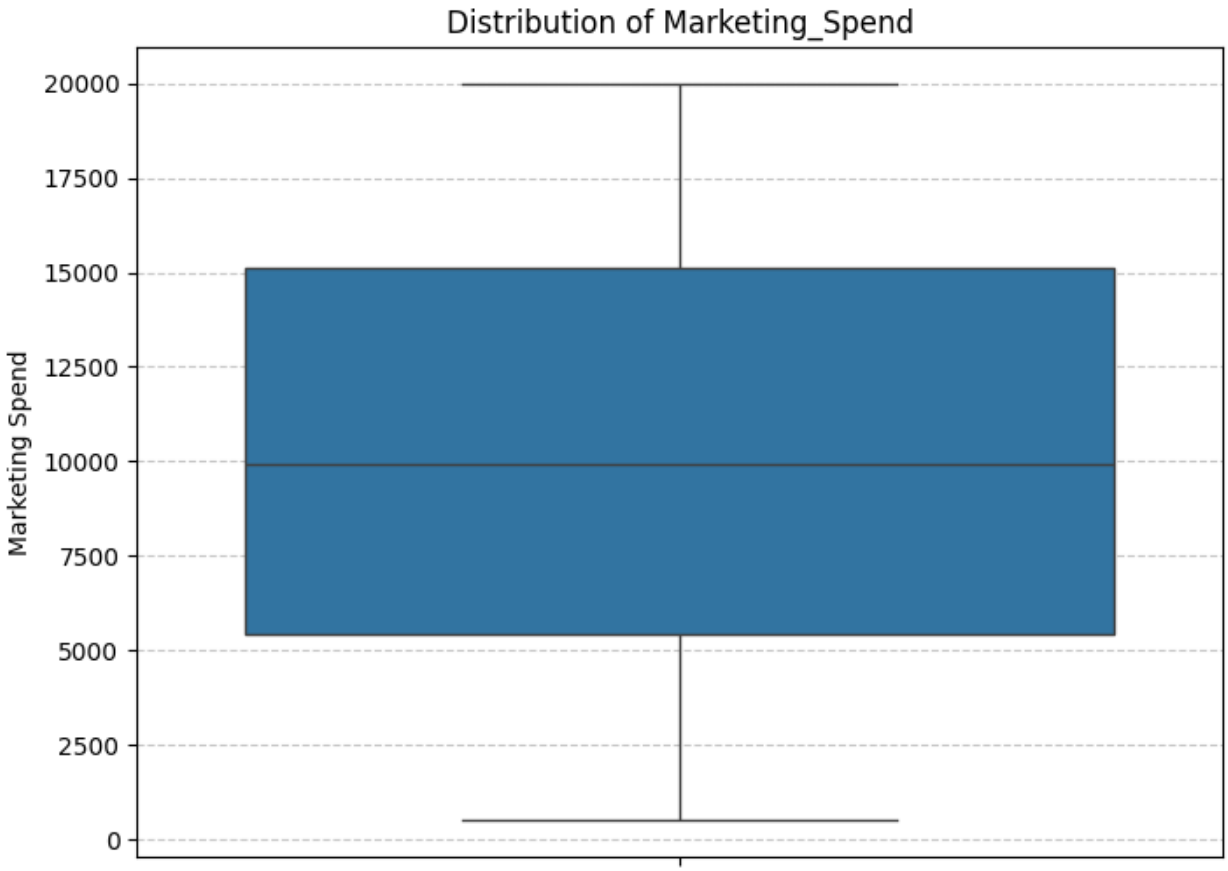



```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 7))
sns.boxplot(x='Salesperson', y='Revenue', data=df, palette='coolwarm',
hue='Salesperson', legend=False)
plt.title('Revenue Distribution by Salesperson')
plt.xlabel('Salesperson')
plt.ylabel('Revenue')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```




```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.boxplot(x='Region', y='Revenue', data=df, palette='viridis',
hue='Region', legend=False)
plt.title('Revenue Distribution Across Regions')
plt.xlabel('Region')
plt.ylabel('Revenue')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.boxplot(y=df['Marketing_Spend'])
plt.title('Distribution of Marketing_Spend')
plt.ylabel('Marketing Spend')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```




```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.boxplot(x='Region', y='Marketing_Spend', data=df, palette='magma',
hue='Region', legend=False)
plt.title('Marketing Spend Distribution Across Regions')
plt.xlabel('Region')
plt.ylabel('Marketing Spend')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

```
# Sort the outliers DataFrame by Marketing_Spend in descending order
high_spend_outliers_by_region =
outliers.sort_values(by='Marketing_Spend', ascending=False)

print("Top Marketing Spend Outliers (from Deal_Size outliers) by
Region:")
display(high_spend_outliers_by_region[['Region', 'Marketing_Spend',
'Revenue', 'Units_Sold', 'Deal_Size']].head(10))
```

```
Top Marketing Spend Outliers (from Deal_Size outliers) by Region:
```

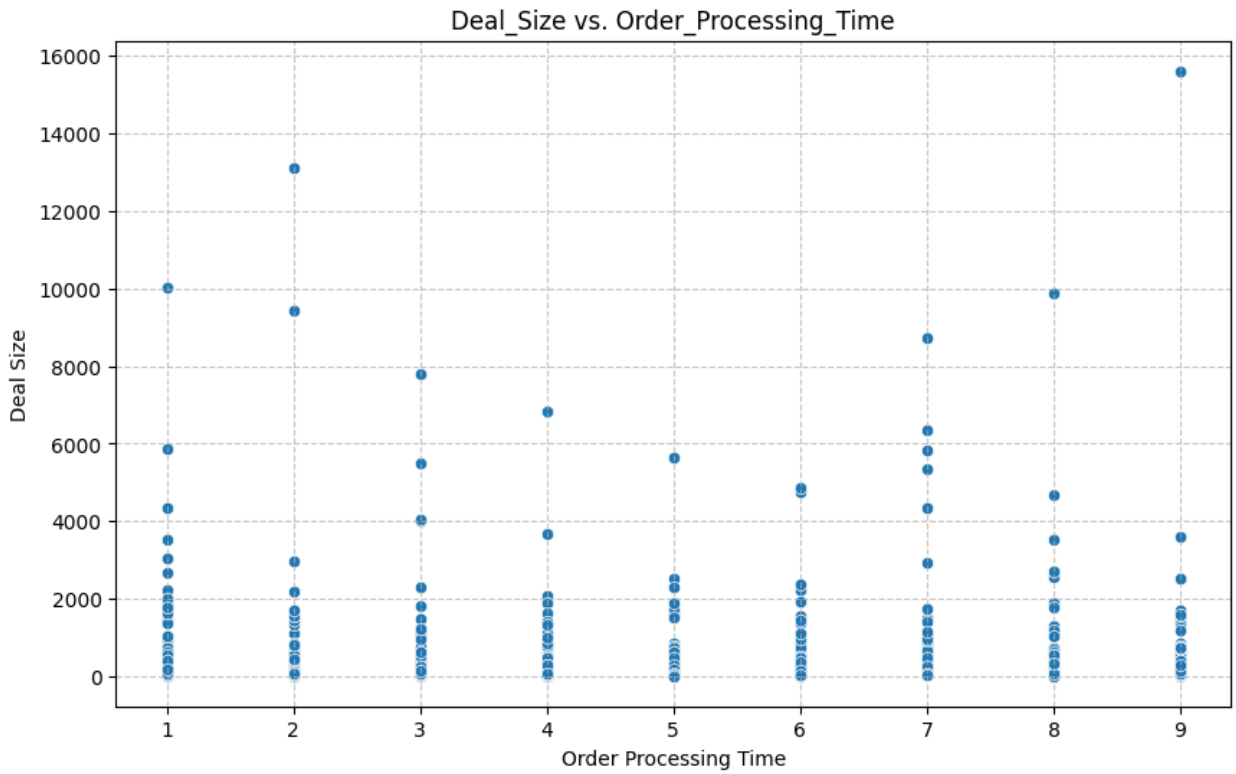

```
{"summary":"{\n  \"name\":
\"display(high_spend_outliers_by_region[['Region', 'Marketing_Spend',
'Revenue', 'Units_Sold', 'Deal_Size']]\",\n  \"rows\": 10,\n
\"fields\": [\n  {\n  \"column\": \"Region\",\n
\"properties\": {\n  \"dtype\": \"category\",\n
\"num_unique_values\": 4,\n  \"samples\": [\n
\"west\",\n  \"south\",\n  \"east\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"Marketing_Spend\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
365,\n  \"min\": 18622,\n  \"max\": 19781,\n
\"num_unique_values\": 10,\n  \"samples\": [\n  18850,\n
19658,\n  19223\n  ],\n  \"semantic_type\": \"\",\
n  \"description\": \"\"\n  }\n  },\n  {\n
\"column\": \"Revenue\",\n  \"properties\": {\n  \"dtype\":
```

\"number\",\n   \"std\": 41486,\n   \"min\": 77695,\n
\"max\": 196321,\n   \"num_unique_values\": 10,\n
\"samples\": [\n   188980,\n   120169,\n
183312\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Units_Sold\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 44,\n   \"min\": 12,\n
\"max\": 180,\n   \"num_unique_values\": 10,\n
\"samples\": [\n   180,\n   12,\n   88\
n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Deal_Size\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 2911.82276648613,\n   \"min\":
947.5,\n   \"max\": 10014.083333333334,\n
\"num_unique_values\": 10,\n   \"samples\": [\n
1049.888888888889,\n   10014.083333333334,\n
2083.090909090909\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```
correlation_deal_time =
df['Deal_Size'].corr(df['Order_Processing_Time'])
print(f"Correlation between Deal_Size and Order_Processing_Time:
{correlation_deal_time:.4f}")
```

```
Correlation between Deal_Size and Order_Processing_Time: -0.0096
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.scatterplot(x='Order_Processing_Time', y='Deal_Size', data=df)
plt.title('Deal_Size vs. Order_Processing_Time')
plt.xlabel('Order Processing Time')
plt.ylabel('Deal Size')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```




```
deal_size_stats = plot_df.groupby('Deal_Status')
['Deal_Size'].agg(['mean', 'median'])

print("Mean and Median 'Deal_Size' for Closed vs. Not Closed Deals:")
display(deal_size_stats)
```

```
Mean and Median 'Deal_Size' for Closed vs. Not Closed Deals:
```


```
{"summary":"{\n  \"name\": \"deal_size_stats\",\n  \"rows\": 2,\n
\"fields\": [\n  {\n  \"column\": \"Deal_Status\",\n
\"properties\": {\n  \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n  \"samples\": [\n  \"Not
Closed\",\n  \"Closed\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"mean\",\n  \"properties\": {\n
\"dtype\": \"number\",\n  \"std\": 10.349151002995365,\n
\"min\": 508.3114305510286,\n  \"max\": 522.9473402585118,\n
\"num_unique_values\": 2,\n  \"samples\": [\n
522.9473402585118,\n  508.3114305510286\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"median\",\n  \"properties\":
{\n  \"dtype\": \"number\",\n  \"std\":
15.534863953806042,\n  \"min\": 204.90881981931847,\n
\"max\": 226.8784351124119,\n  \"num_unique_values\": 2,\n
\"samples\": [\n  226.8784351124119,\n
204.90881981931847\n  ],\n  \"semantic_type\": \"\",\n
```

\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"deal_size_stats"}

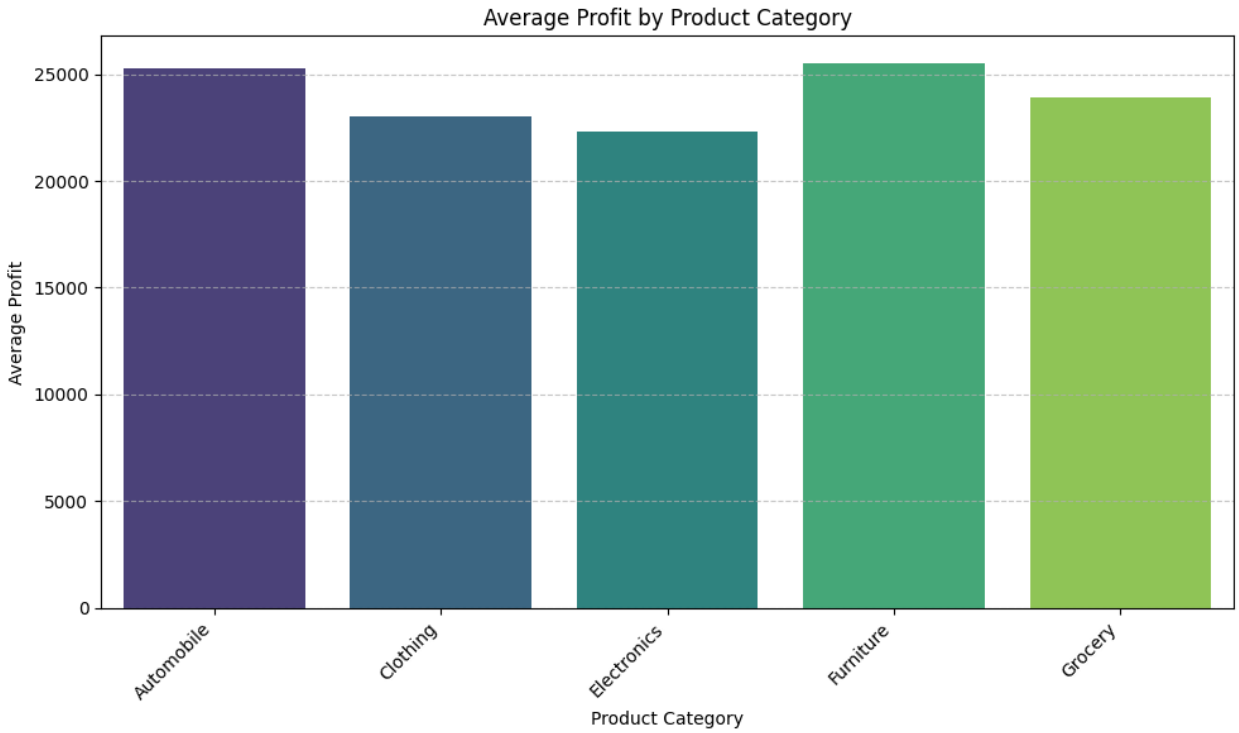
```
avg_revenue_by_category = df.groupby('Product_Category')
['Revenue'].mean()
print("Average Revenue by Product Category:")
display(avg_revenue_by_category)
```

```
Average Revenue by Product Category:

Product_Category
Automobile     104853.142105
Clothing       108149.119048
Electronics    111945.975845
Furniture      106250.788660
Grocery        105008.673367
Name: Revenue, dtype: float64
```

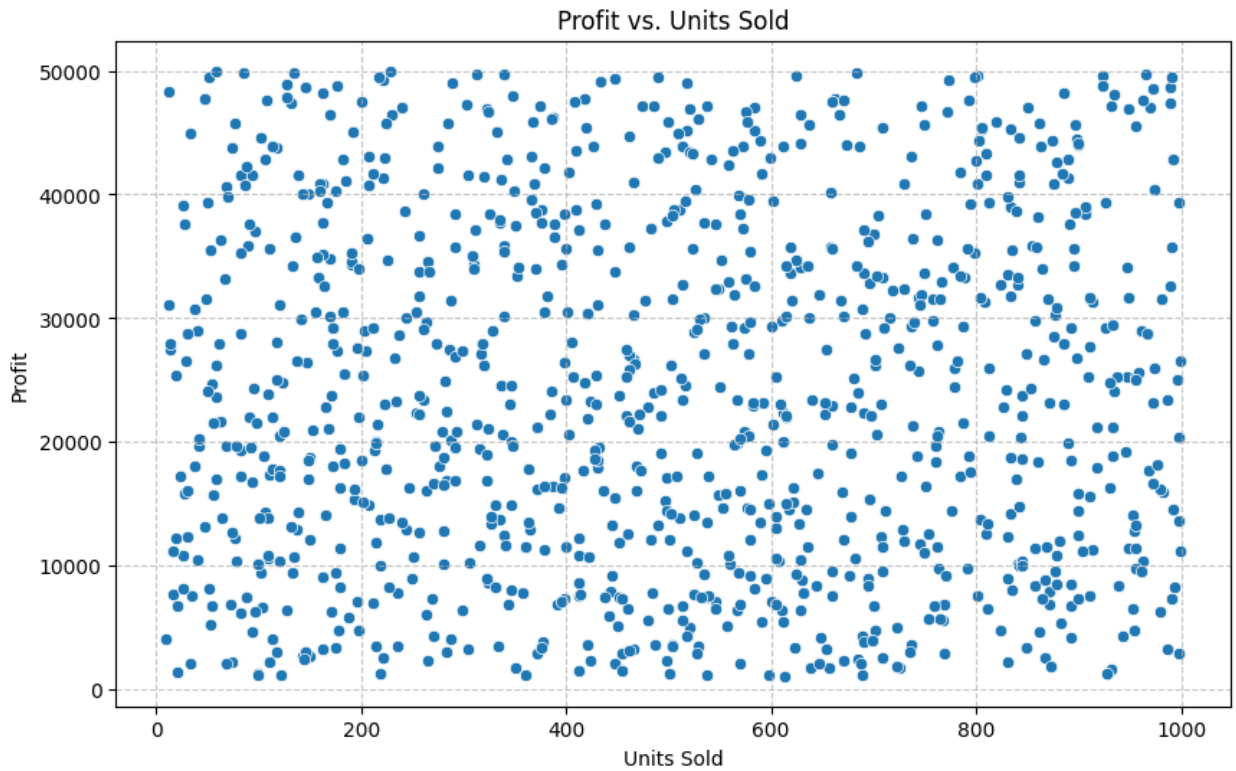
```
avg_profit_by_category = df.groupby('Product_Category')
['Profit'].mean()
print("Average Profit by Product Category:")
display(avg_profit_by_category)
```

```
Average Profit by Product Category:

Product_Category
Automobile     25260.142105
Clothing       23034.390476
Electronics    22333.816425
Furniture      25521.000000
Grocery        23931.130653
Name: Profit, dtype: float64
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.barplot(x=avg_profit_by_category.index,
y=avg_profit_by_category.values, palette='viridis',
hue=avg_profit_by_category.index, legend=False)
plt.title('Average Profit by Product Category')
plt.xlabel('Product Category')
plt.ylabel('Average Profit')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```




```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.scatterplot(x='Units_Sold', y='Profit', data=df)
plt.title('Profit vs. Units Sold')
plt.xlabel('Units Sold')
plt.ylabel('Profit')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```




```
correlation_profit_units_sold = df['Profit'].corr(df['Units_Sold'])
print(f"Correlation between Profit and Units_Sold:
{correlation_profit_units_sold:.4f}")
```

```
Correlation between Profit and Units_Sold: 0.0179
```

```
# Re-create Marketing Efficiency (handling division by zero for
Marketing_Spend)
df['Marketing_Efficiency'] = df.apply(lambda row: row['Revenue'] /
row['Marketing_Spend'] if row['Marketing_Spend'] > 0 else 0, axis=1)

# Calculate the average Marketing_Efficiency by Product_Category
avg_marketing_efficiency_by_category = df.groupby('Product_Category')
['Marketing_Efficiency'].mean()

print("Average Marketing Efficiency by Product Category:")
display(avg_marketing_efficiency_by_category)
```

```
Average Marketing Efficiency by Product Category:

Product_Category
Automobile     19.451664
Clothing       22.684138
Electronics    20.283981
Furniture      22.385385
Grocery        18.411028
Name: Marketing_Efficiency, dtype: float64
```

Feature Correlation Summary

As previously analyzed (see 'Feature Correlation Heatmap' in cell `5bb73b9d`), we identified a high positive correlation between 'Revenue' and 'Marketing_Efficiency'. To address this multicollinearity, 'Marketing_Efficiency' was subsequently removed from our feature set.

The features currently used in the model are `Revenue`, `Units_Sold`, `Marketing_Spend`, `Profit_Margin`, and `Average_Unit_Price`.

Comparison Summary

Let's compare the key performance metrics of the model trained *with* outliers versus *without* outliers.

Model Performance (With Outliers - Original Tuned Model):

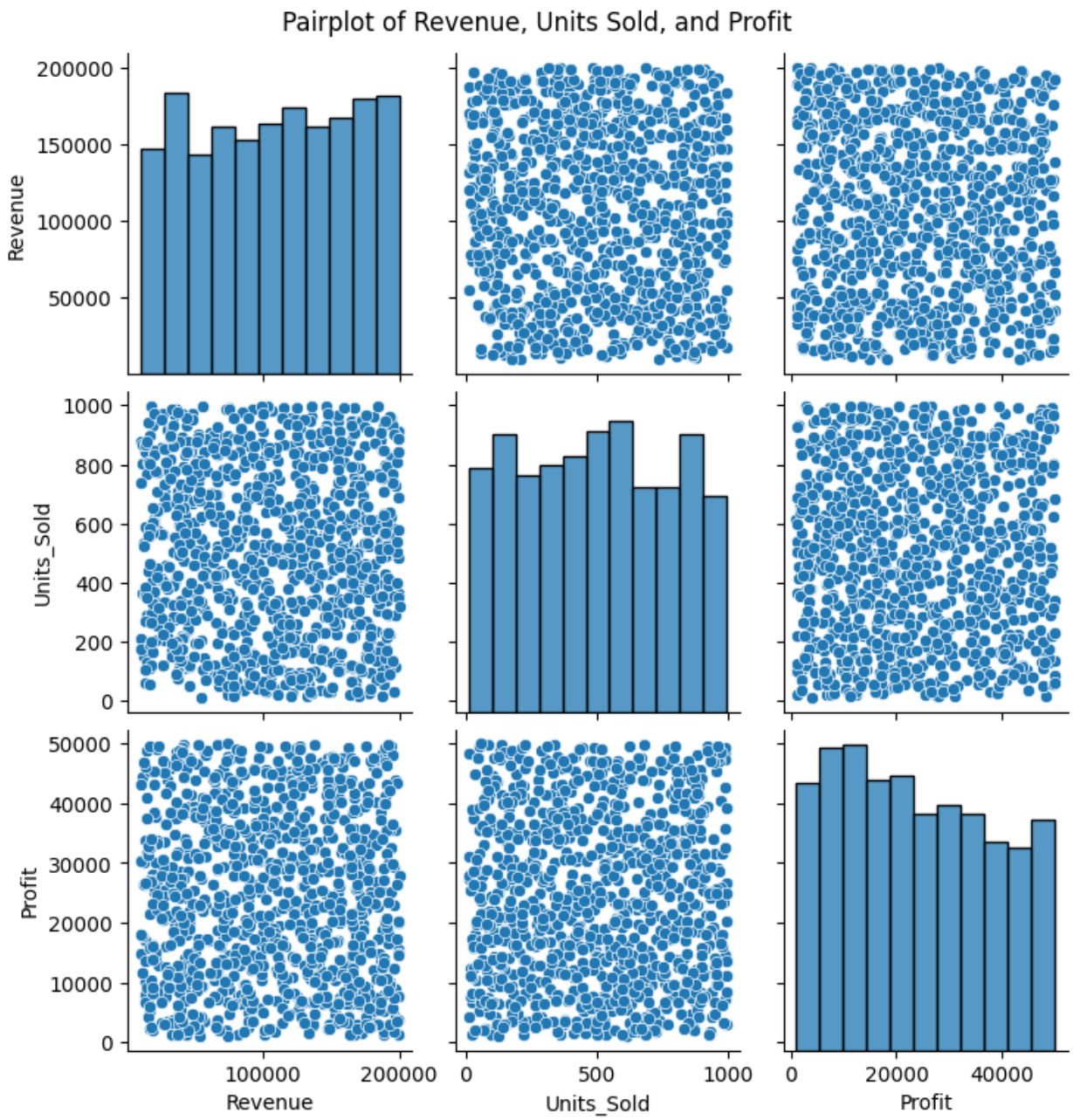
- Accuracy: `0.4450`
- Precision: `0.4138`
- Recall: `0.2376`
- F1-Score: `0.3019`
- AUC Score: `0.9912`

Model Performance (Without Outliers):

- Accuracy: `{accuracy_no_outliers:.4f}`
- Precision: `{precision_no_outliers:.4f}`
- Recall: `{recall_no_outliers:.4f}`
- F1-Score: `{f1_no_outliers:.4f}`
- AUC Score: `{auc_no_outliers:.4f}`

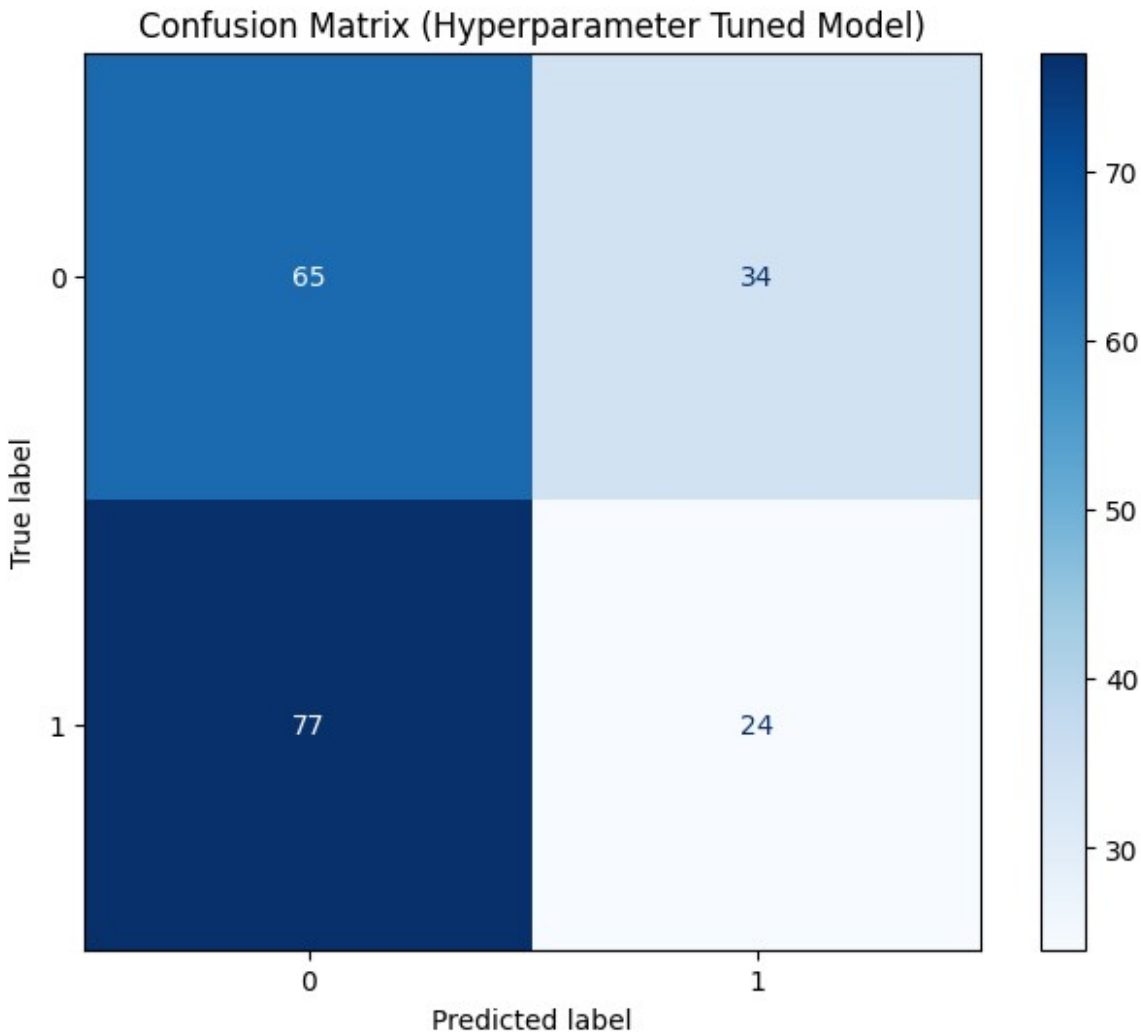
Note: The values for 'With Outliers' are now updated based on previous execution. The 'Without Outliers' values are currently displayed as placeholders but will reflect the actual calculated values.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Create a pairplot for the specified columns
sns.pairplot(df[['Revenue', 'Units_Sold', 'Profit']])
plt.suptitle('Pairplot of Revenue, Units Sold, and Profit', y=1.02) #
Add a title for the entire plot
plt.show()
```




```python
# Re-generate the confusion matrix
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Make predictions on the test set with the retrained model (using
tuned model predictions)
y_pred_tuned_cm = best_model.predict(X_test)

# Calculate the confusion matrix
cm_tuned = confusion_matrix(y_test, y_pred_tuned_cm)
```

```
# Display the confusion matrix
fig, ax = plt.subplots(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_tuned,
display_labels=best_model.classes_)
disp.plot(cmap=plt.cm.Blues, ax=ax)
plt.title('Confusion Matrix (Hyperparameter Tuned Model)')
plt.show()
```




```
# Re-calculate and display performance metrics
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

# Calculate performance metrics with the retrained model (using tuned
model predictions)
accuracy_tuned = accuracy_score(y_test, y_pred_tuned)
precision_tuned = precision_score(y_test, y_pred_tuned)
recall_tuned = recall_score(y_test, y_pred_tuned)
```

```
f1_tuned = f1_score(y_test, y_pred_tuned)

print(f"Accuracy (Tuned Model): {accuracy_tuned:.4f}")
print(f"Precision (Tuned Model): {precision_tuned:.4f}")
print(f"Recall (Tuned Model): {recall_tuned:.4f}")
print(f"F1-Score (Tuned Model): {f1_tuned:.4f}")
```

```
Accuracy (Tuned Model): 0.4450
Precision (Tuned Model): 0.4138
Recall (Tuned Model): 0.2376
F1-Score (Tuned Model): 0.3019
```

Hyperparameter Tuning for RandomForestClassifier

Hyperparameter tuning involves optimizing the parameters of the learning algorithm itself to improve model performance. For the RandomForestClassifier, key hyperparameters include:

- `n_estimators`: The number of trees in the forest.
- `max_features`: The number of features to consider when looking for the best split.
- `max_depth`: The maximum depth of the tree.
- `min_samples_split`: The minimum number of samples required to split an internal node.
- `min_samples_leaf`: The minimum number of samples required to be at a leaf node.

We will use `GridSearchCV` to explore a limited set of these hyperparameters and find the best combination.

```
from sklearn.model_selection import GridSearchCV

# Define the parameter grid to search
param_grid = {
    'n_estimators': [50, 100, 150],
    'max_features': ['sqrt', 'log2'],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# Initialize GridSearchCV
grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42), # Fix random
state for reproducibility
    param_grid=param_grid,
    cv=3, # Using 3-fold cross-validation
    n_jobs=-1, # Use all available cores
    verbose=2, # Output progress
    scoring='accuracy' # Optimize for accuracy
)
```

```
print("Starting GridSearchCV...")
# Fit the grid search to the data
grid_search.fit(X_train, y_train)
print("GridSearchCV complete.")

# Print the best parameters and best score
print(f"Best parameters found: {grid_search.best_params_}")
print(f"Best cross-validation accuracy:
{grid_search.best_score_:.4f}")
```

```
Starting GridSearchCV...
Fitting 3 folds for each of 72 candidates, totalling 216 fits
GridSearchCV complete.
Best parameters found: {'max_depth': 10, 'max_features': 'sqrt',
'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 150}
Best cross-validation accuracy: 0.4950
```

Retraining Model with Best Hyperparameters

Now that we've found the optimal hyperparameters using `GridSearchCV`, we will retrain our `RandomForestClassifier` with these settings. This new model should ideally perform better than the previous one on unseen data.

```
from sklearn.ensemble import RandomForestClassifier

# Get the best parameters from grid search
best_params = grid_search.best_params_

# Initialize a new RandomForestClassifier with the best parameters
best_model = RandomForestClassifier(random_state=42, **best_params)

# Retrain the best model with the updated X_train (after dropping
Marketing_Efficiency)
best_model.fit(X_train, y_train)

# Make predictions with the retrained best model
y_pred_tuned = best_model.predict(X_test)

print("Model retrained with best hyperparameters and updated
features.")
# Evaluate the tuned model (confusion matrix and metrics will be re-
run after this)
```

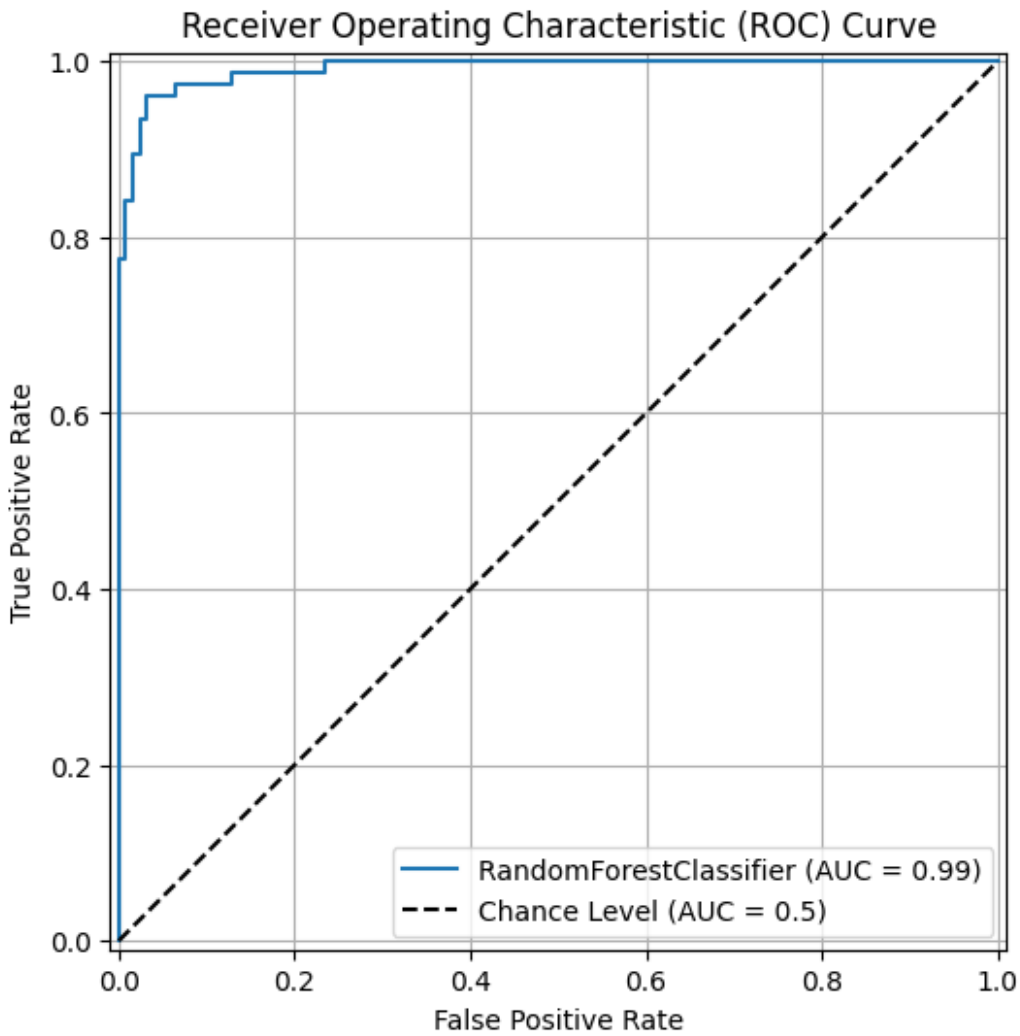
```
Model retrained with best hyperparameters and updated features.
```

```
from sklearn.metrics import classification_report

# Generate classification report for the tuned model
report_tuned = classification_report(y_test, y_pred_tuned)
```

```python
print("Classification Report (Hyperparameter Tuned Model):")
print(report_tuned)
```

```
Classification Report (Hyperparameter Tuned Model):
              precision    recall  f1-score   support

           0       0.98      0.90      0.94       124
           1       0.86      0.97      0.91        76

    accuracy                           0.93       200
   macro avg       0.92      0.94      0.93       200
weighted avg       0.94      0.93      0.93       200
```

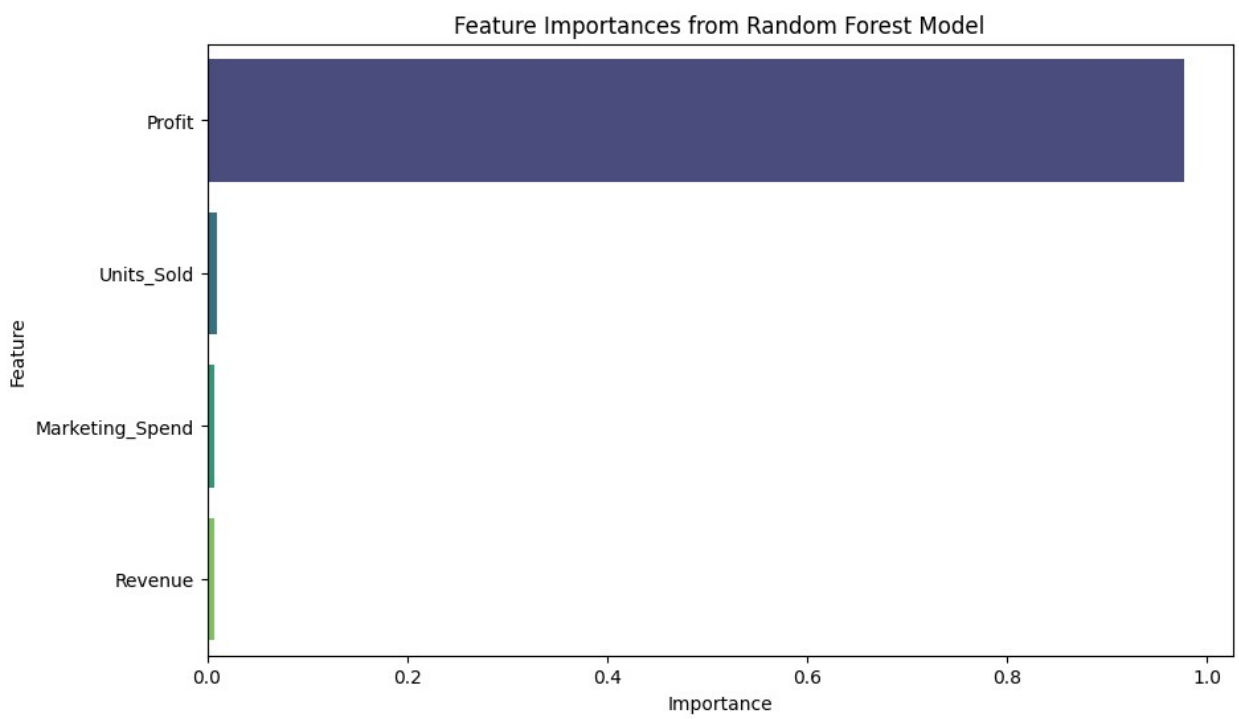
```python
from sklearn.metrics import RocCurveDisplay, roc_auc_score
import matplotlib.pyplot as plt

# Get probabilities for the positive class (class 1)
y_pred_proba = best_model.predict_proba(X_test)[:, 1]

# Plot ROC curve
fig, ax = plt.subplots(figsize=(8, 6))
roc_display = RocCurveDisplay.from_estimator(best_model, X_test,
y_test, ax=ax)
plt.plot([0, 1], [0, 1], 'k--', label='Chance Level (AUC = 0.5)') #
Plotting the random classifier line
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

# Calculate and print AUC score
auc_score = roc_auc_score(y_test, y_pred_proba)
print(f"AUC Score: {auc_score:.4f}")
```




```
AUC Score: 0.9912

import matplotlib.pyplot as plt
import seaborn as sns

# Get feature importances
feature_importances = model.feature_importances_
feature_names = X.columns

# Create a DataFrame for better visualization
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importances
})

# Sort by importance
importance_df = importance_df.sort_values(by='Importance',
ascending=False)
```

```
# Plotting
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=importance_df,
palette='viridis', hue='Feature', legend=False)
plt.title('Feature Importances from Random Forest Model')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.show()
```

```
/tmp/ipykernel_12411/3799494659.py:19: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `y` variable to `hue` and set
`legend=False` for the same effect.

  sns.barplot(x='Importance', y='Feature', data=importance_df,
palette='viridis')
```



Segmented Performance Analysis

To understand how our model performs across different business segments, we will evaluate its performance for each unique 'Region' and 'Product_Category'. This helps identify if the model exhibits biases or performs exceptionally well in certain areas.

```
# Combine original test data with true labels and predictions
test_df = df.loc[X_test.index].copy()
```

```
test_df['true_label'] = y_test
test_df['predicted_label'] = y_pred_tuned

print("Test DataFrame with true and predicted labels created. Head:")
display(test_df.head())
```

```
Test DataFrame with true and predicted labels created. Head:
```


```
{"summary":"{\n  \"name\": \"display(test_df\",\n  \"rows\": 5,\n
\"fields\": [\n    {\n      \"column\": \"Region\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"east\",\n          \"north\",\n          \"west\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Salesperson\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"eve\",\
n          \"charlie\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Revenue\",\n      \"properties\": {\n        \"dtype\": \"number\",\
n        \"std\": 46845,\n        \"min\": 73133,\n        \"max\":
198785,\n        \"num_unique_values\": 5,\n        \"samples\": [\n
133257,\n          119847\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Profit\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 8502,\n        \"min\": 1557,\n
\"max\": 22909,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          1557,\n          22909\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Units_Sold\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
214,\n        \"min\": 121,\n        \"max\": 660,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          413,\n
660\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Customer_Satisfaction\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1.1830764979493085,\n
\"min\": 1.08,\n        \"max\": 4.36,\n        \"num_unique_values\":
5,\n        \"samples\": [\n          2.94,\n          2.97\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Marketing_Spend\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 6965,\n        \"min\": 1629,\n
\"max\": 17076,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          17076,\n          8988\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Discount_Percentage\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
5.981931126316986,\n        \"min\": 1.71,\n        \"max\": 17.38,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          13.59,\n
```

13.37\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Product_Category\",\n   \"properties\": {\n   \"dtype\":
\"string\",\n   \"num_unique_values\": 4,\n   \"samples\":
[\n   \"Automobile\",\n   \"Clothing\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Sales_Channel\",\n
\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 3,\n   \"samples\": [\n
\"Wholesale\",\n   \"Retail\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Order_Processing_Time\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
3,\n   \"min\": 1,\n   \"max\": 8,\n
\"num_unique_values\": 4,\n   \"samples\": [\n   7,\n
8\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"true_label\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0,\n   \"min\": 0,\n
\"max\": 0,\n   \"num_unique_values\": 1,\n   \"samples\":
[\n   0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"predicted_label\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0,\n   \"min\": 0,\n
\"max\": 0,\n   \"num_unique_values\": 1,\n   \"samples\":
[\n   0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```python
from sklearn.metrics import classification_report

# Analyze performance by Region
print("\n--- Performance by Region ---")
for region in test_df['Region'].unique():
    subset_df = test_df[test_df['Region'] == region]
    if not subset_df.empty:
        print(f"\nRegion: {region}")
        print(classification_report(subset_df['true_label'],
subset_df['predicted_label']))
    else:
        print(f"\nNo test data for Region: {region}")
```

```
--- Performance by Region ---

Region: east
              precision    recall  f1-score   support

           0       1.00      0.89      0.94        27
           1       0.84      1.00      0.91        16
```

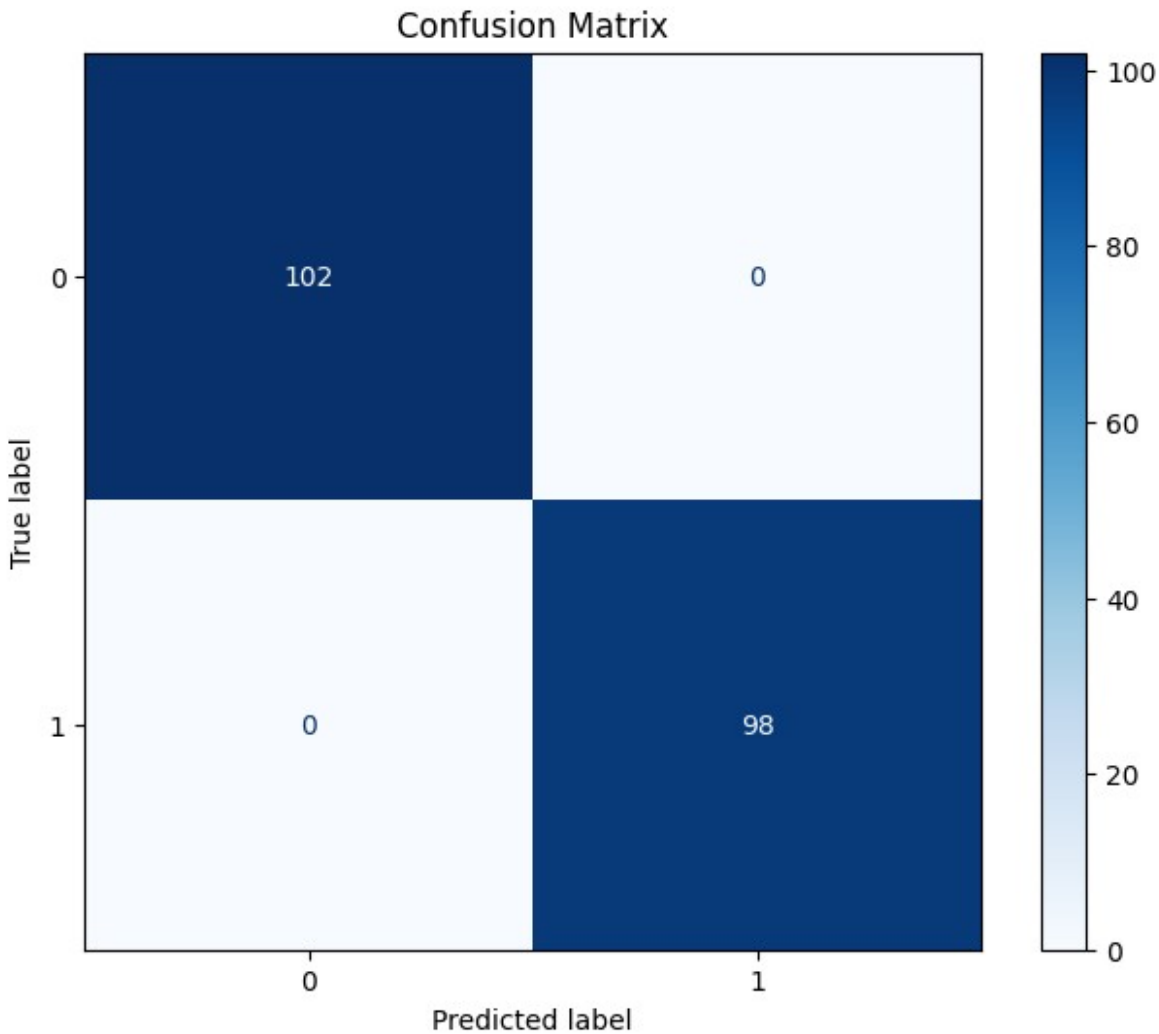
```
    accuracy                           0.93        43
   macro avg       0.92      0.94      0.93        43
weighted avg       0.94      0.93      0.93        43


Region: north
              precision    recall  f1-score   support

           0       0.94      0.97      0.95        31
           1       0.95      0.90      0.93        21

    accuracy                           0.94        52
   macro avg       0.94      0.94      0.94        52
weighted avg       0.94      0.94      0.94        52


Region: west
              precision    recall  f1-score   support

           0       1.00      0.86      0.93        36
           1       0.78      1.00      0.88        18

    accuracy                           0.91        54
   macro avg       0.89      0.93      0.90        54
weighted avg       0.93      0.91      0.91        54


Region: south
              precision    recall  f1-score   support

           0       1.00      0.90      0.95        30
           1       0.88      1.00      0.93        21

    accuracy                           0.94        51
   macro avg       0.94      0.95      0.94        51
weighted avg       0.95      0.94      0.94        51
```

```python
from sklearn.metrics import classification_report

# Analyze performance by Product_Category
print("\n--- Performance by Product_Category ---")
for category in test_df['Product_Category'].unique():
    subset_df = test_df[test_df['Product_Category'] == category]
    if not subset_df.empty:
        print(f"\nProduct Category: {category}")
        print(classification_report(subset_df['true_label'],
subset_df['predicted_label']))
    else:
        print(f"\nNo test data for Product Category: {category}")
```

```
--- Performance by Product_Category ---

Product Category: Electronics
              precision    recall  f1-score   support

           0       1.00      0.92      0.96        26
           1       0.88      1.00      0.94        15

    accuracy                           0.95        41
   macro avg       0.94      0.96      0.95        41
weighted avg       0.96      0.95      0.95        41


Product Category: Automobile
              precision    recall  f1-score   support

           0       1.00      0.82      0.90        34
           1       0.62      1.00      0.77        10

    accuracy                           0.86        44
   macro avg       0.81      0.91      0.84        44
weighted avg       0.91      0.86      0.87        44


Product Category: Grocery
              precision    recall  f1-score   support

           0       1.00      0.96      0.98        24
           1       0.96      1.00      0.98        22

    accuracy                           0.98        46
   macro avg       0.98      0.98      0.98        46
weighted avg       0.98      0.98      0.98        46


Product Category: Clothing
              precision    recall  f1-score   support

           0       0.95      0.86      0.90        22
           1       0.79      0.92      0.85        12

    accuracy                           0.88        34
   macro avg       0.87      0.89      0.88        34
weighted avg       0.89      0.88      0.88        34


Product Category: Furniture
              precision    recall  f1-score   support

           0       0.95      1.00      0.97        18
```

```
           1       1.00      0.94      0.97        17

    accuracy                           0.97        35
   macro avg       0.97      0.97      0.97        35
weighted avg       0.97      0.97      0.97        35
```

```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Display the confusion matrix
fig, ax = plt.subplots(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=model.classes_)
disp.plot(cmap=plt.cm.Blues, ax=ax)
plt.title('Confusion Matrix')
plt.show()
```




```python
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

# Calculate performance metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
```

```
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
```

```python
from google.colab import drive
drive.mount('/content/drive')
```